Chapter 11

Inheritance
&
Polymorphism

Motivations

When developing a game with different types of **characters**: **warriors**, **mages**, and **archers** —each character has **common attributes** like health, attack power and movement speed but also possesses **unique abilities**.

*How to reduce **redundancy** and improve **maintainability?***

Motivations

When developing a game with different types of **characters**: **warriors**, **mages**, and **archers**, each character has **common attributes** like health, attack power











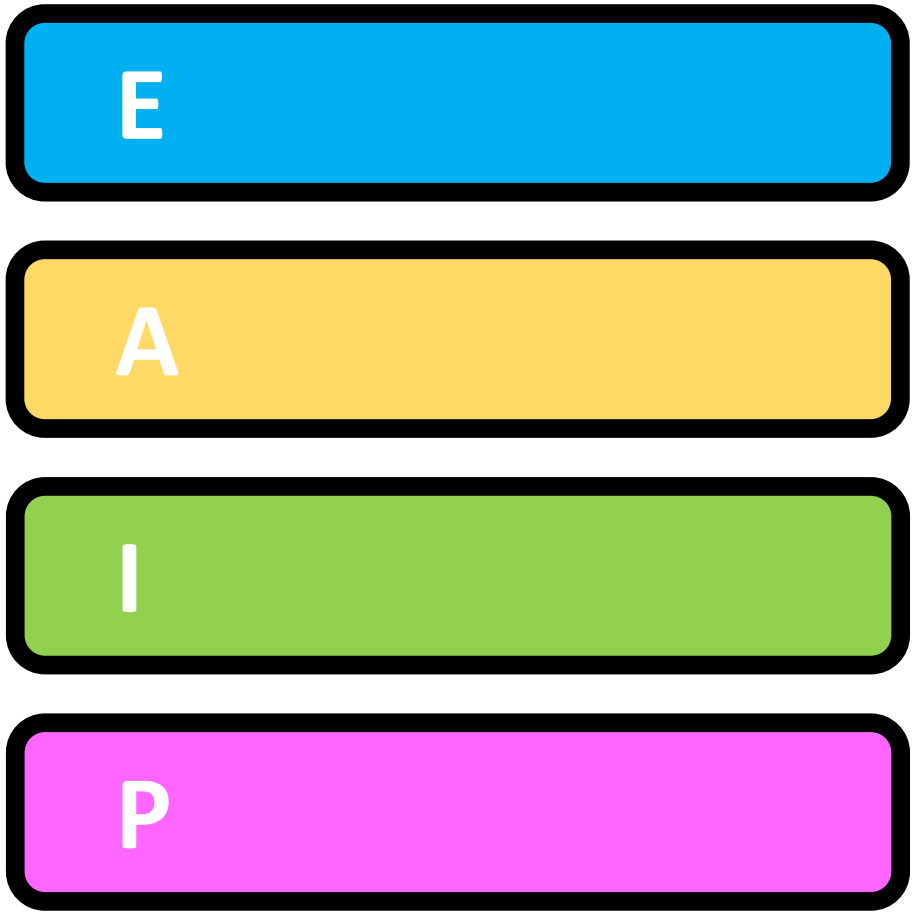

Objectives

11.2 To invoke the superclass's constructors and methods using the **`super`** keyword (§11.3).
11.3 To override instance methods in the subclass (§11.4).
11.4 To distinguish differences between overriding and overloading (§11.5).
11.5 To explore the **`toString()`** method in the **`Object`** class (§11.6).
11.6 To discover polymorphism and dynamic binding (§§11.7–11.8).
11.7 To describe casting and explain why explicit downcasting is necessary (§11.9).
11.8 To explore the **`equals`** method in the **`Object`** class (§11.10).
11.9 To store, retrieve, and manipulate objects in an `ArrayList` (§11.11).
11.10 To construct an array list from an array, to sort and shuffle a list, and to obtain max and min element from a list (§11.12).
11.11 To implement a **`Stack`** class using **`ArrayList`** (§11.13).
11.12 To enable data and methods in a superclass accessible from subclasses using the **`protected`** visibility modifier (§11.14).
11.13 To prevent class extending and method overriding using the **`final`** modifier (§11.14).
11.14 To automatically generate boilerplate code using Lombok (§11.16).

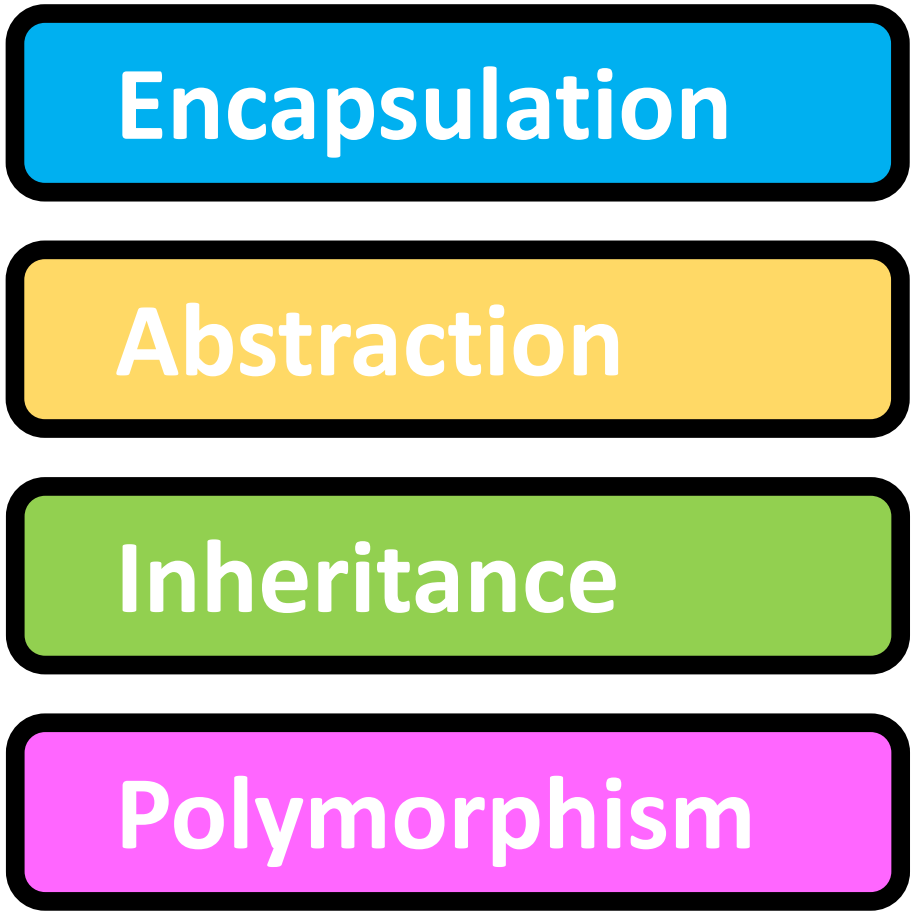
Objectives

- Inheritance
- Polymorphism
  - override vs. overload

Object-Oriented Programming (OOP)



Object-Oriented Programming (OOP)



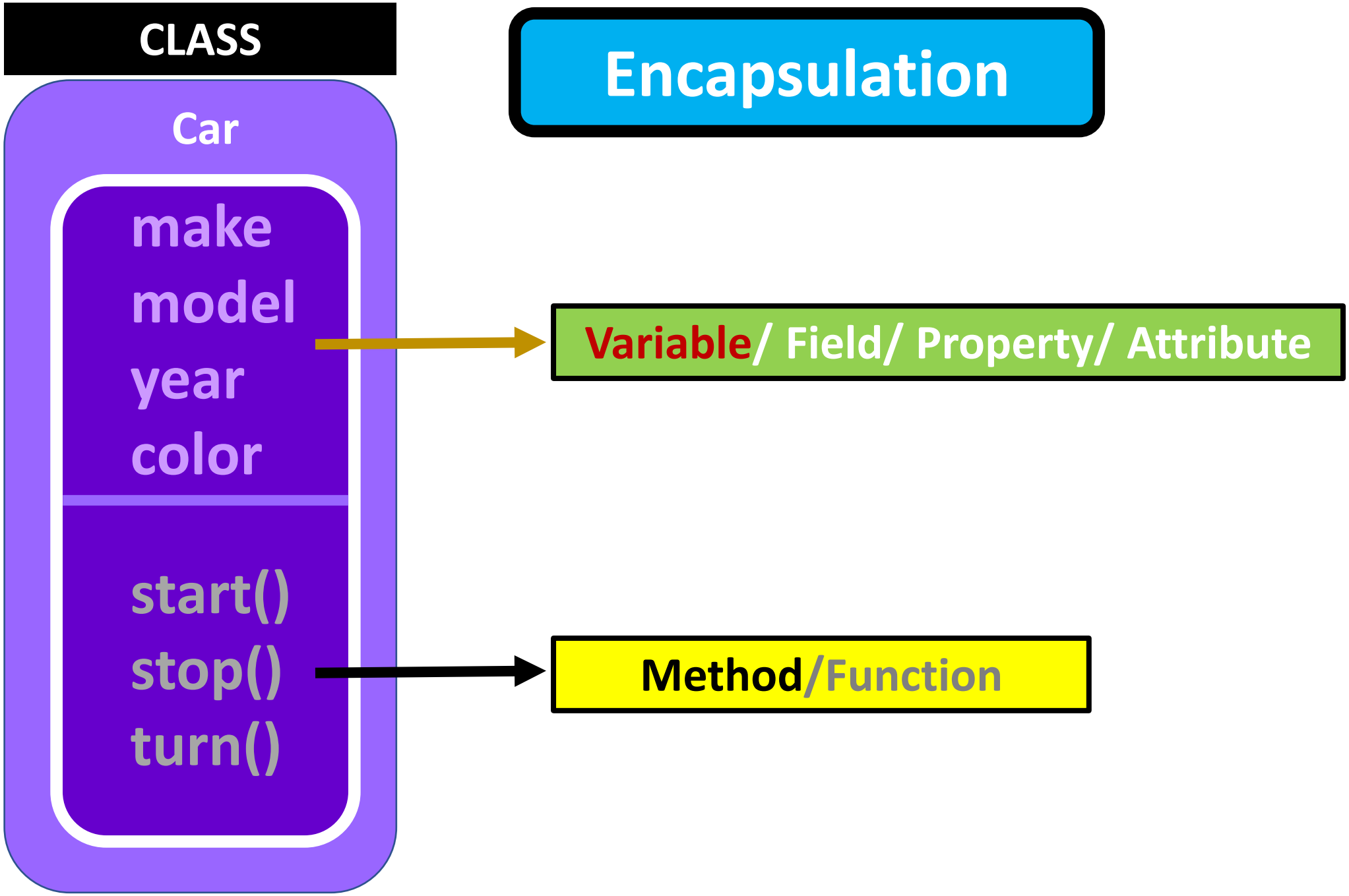
Encapsulation

make
model
year
color

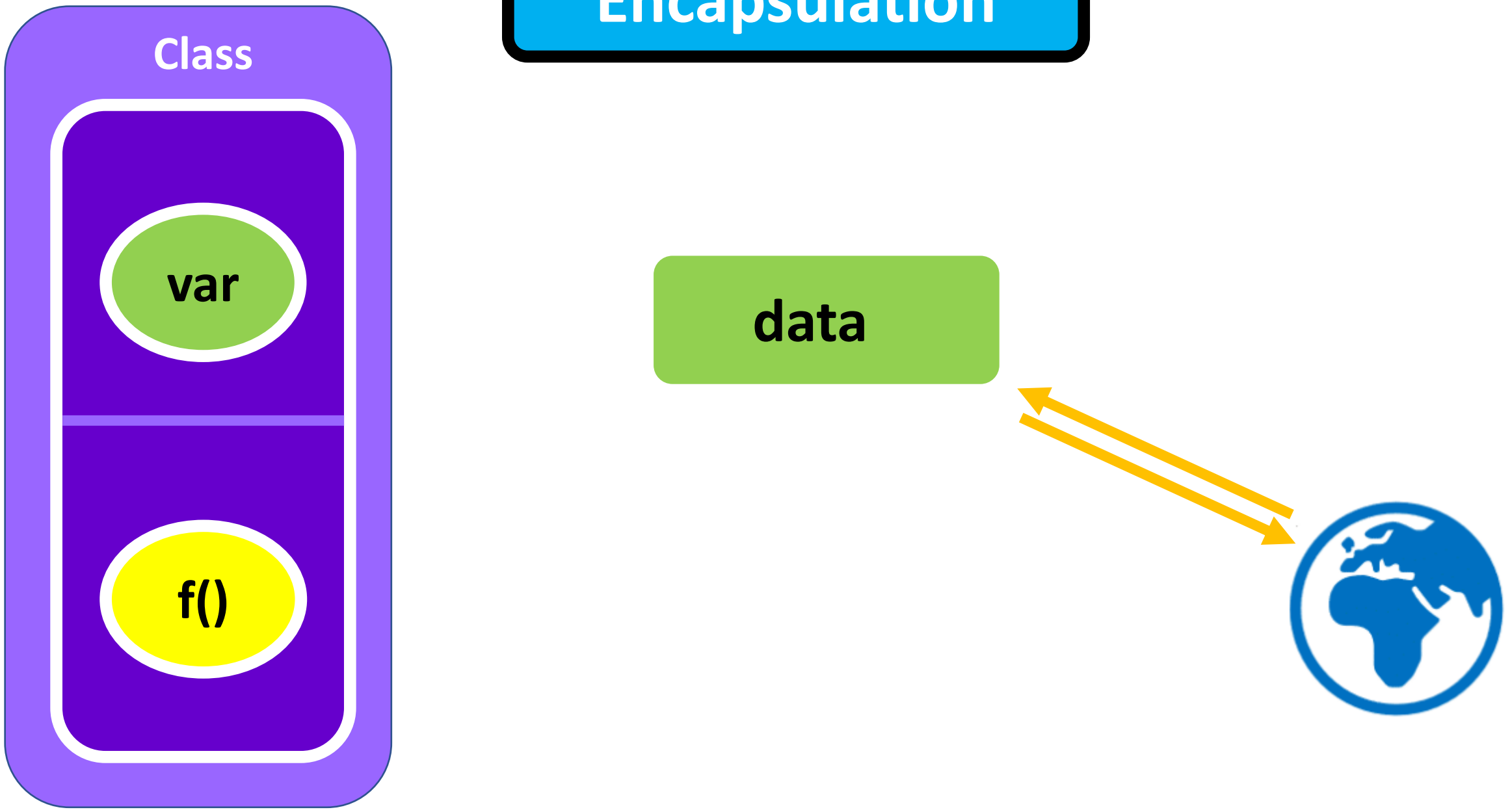
Variable/ Field/ Property/ Attribute

start()
stop()
turn()

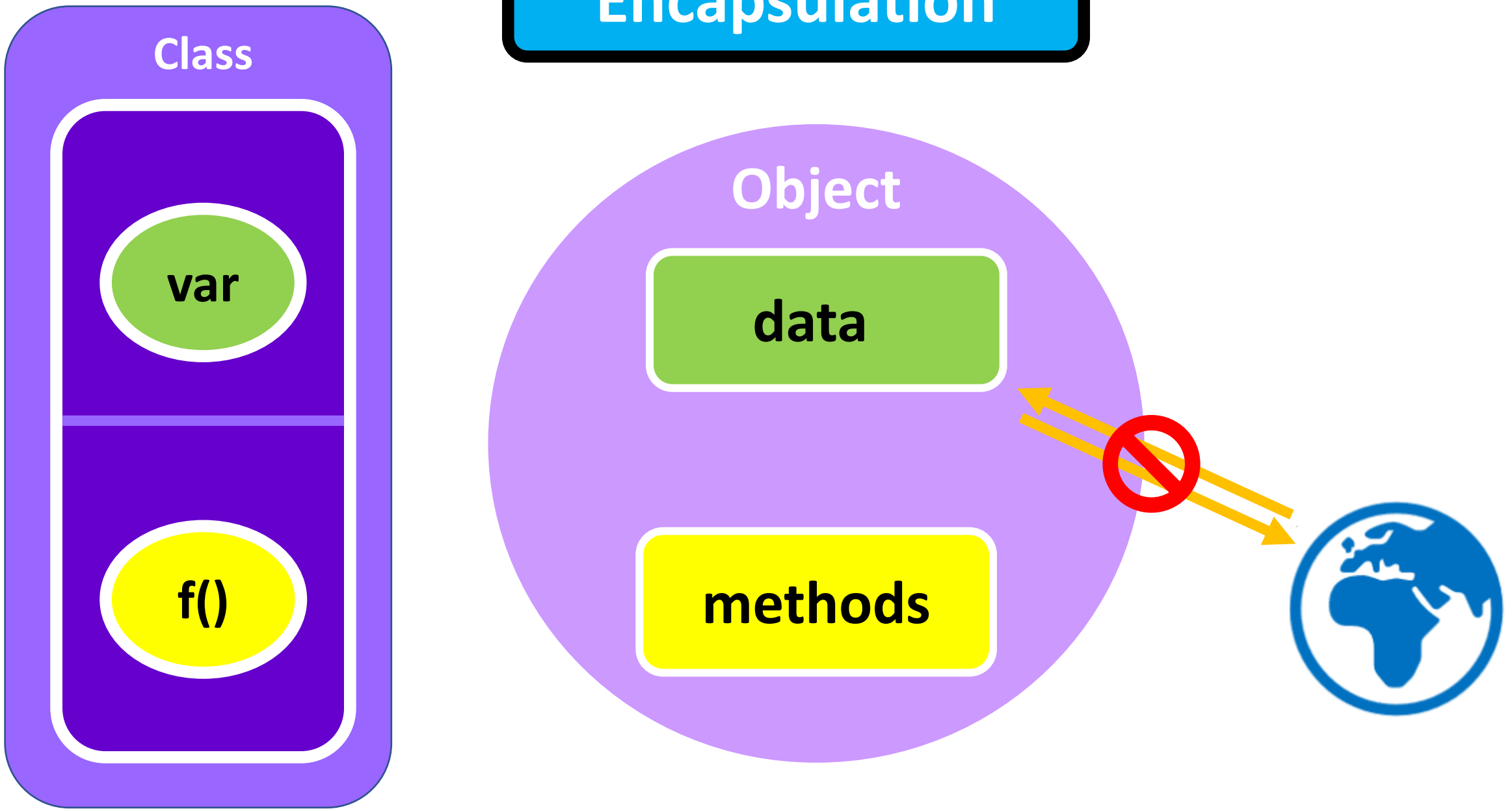
Method/Function


CLASS
Car
make
model
year
color
start()
stop()
turn()
Encapsulation
Variable/ Field/ Property/ Attribute
Method/Function

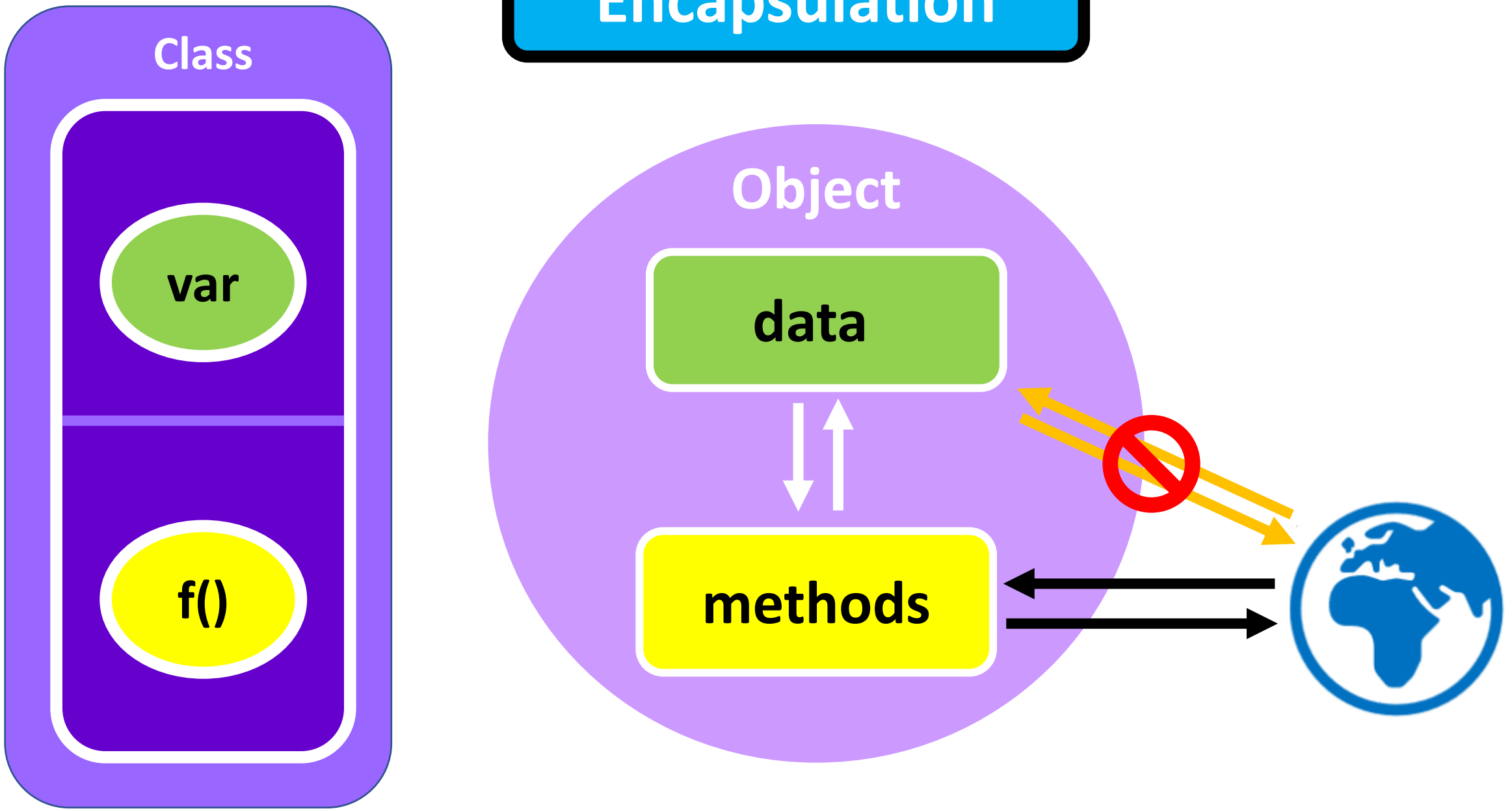
Encapsulation



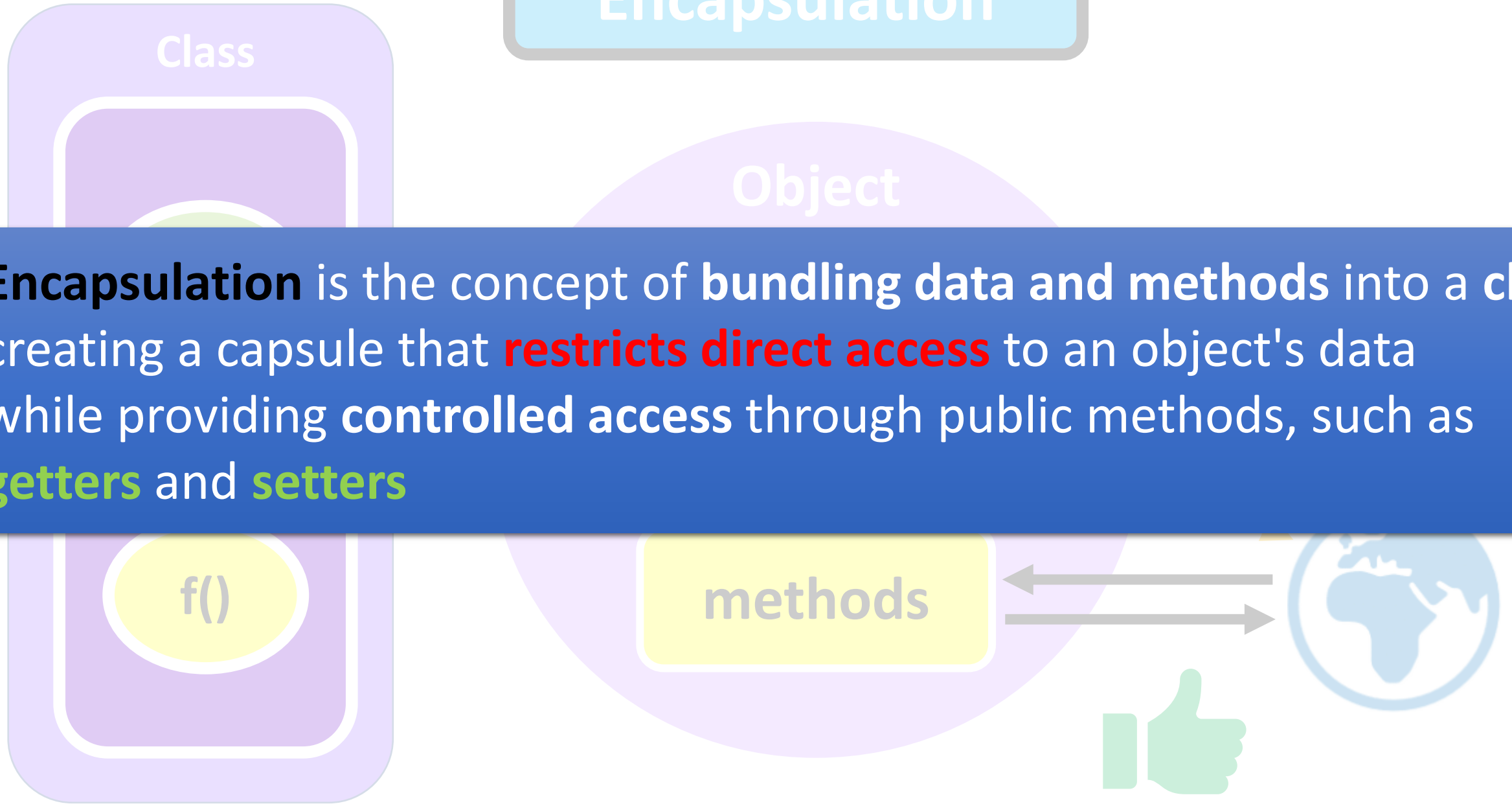
Encapsulation



Encapsulation



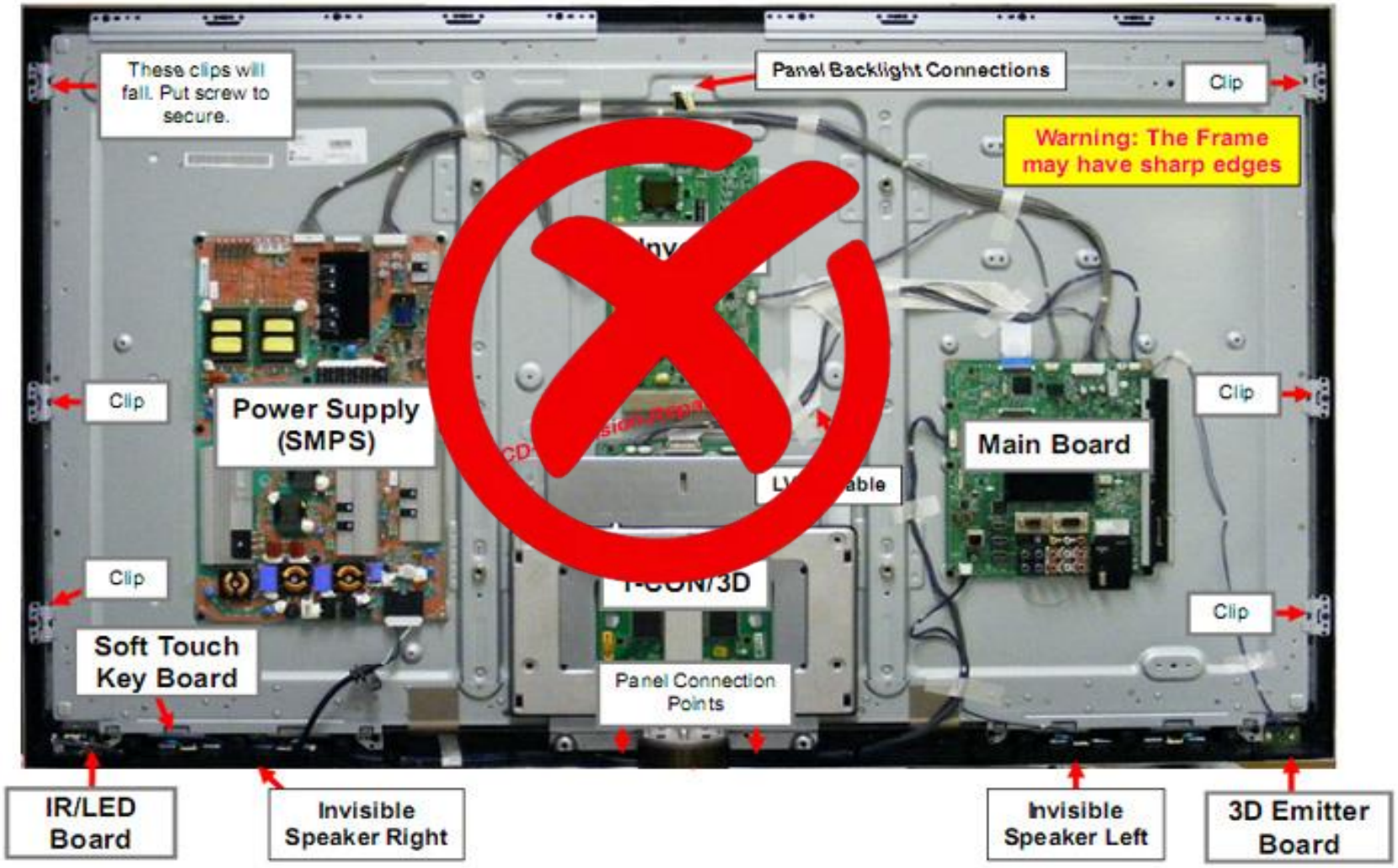
Encapsulation



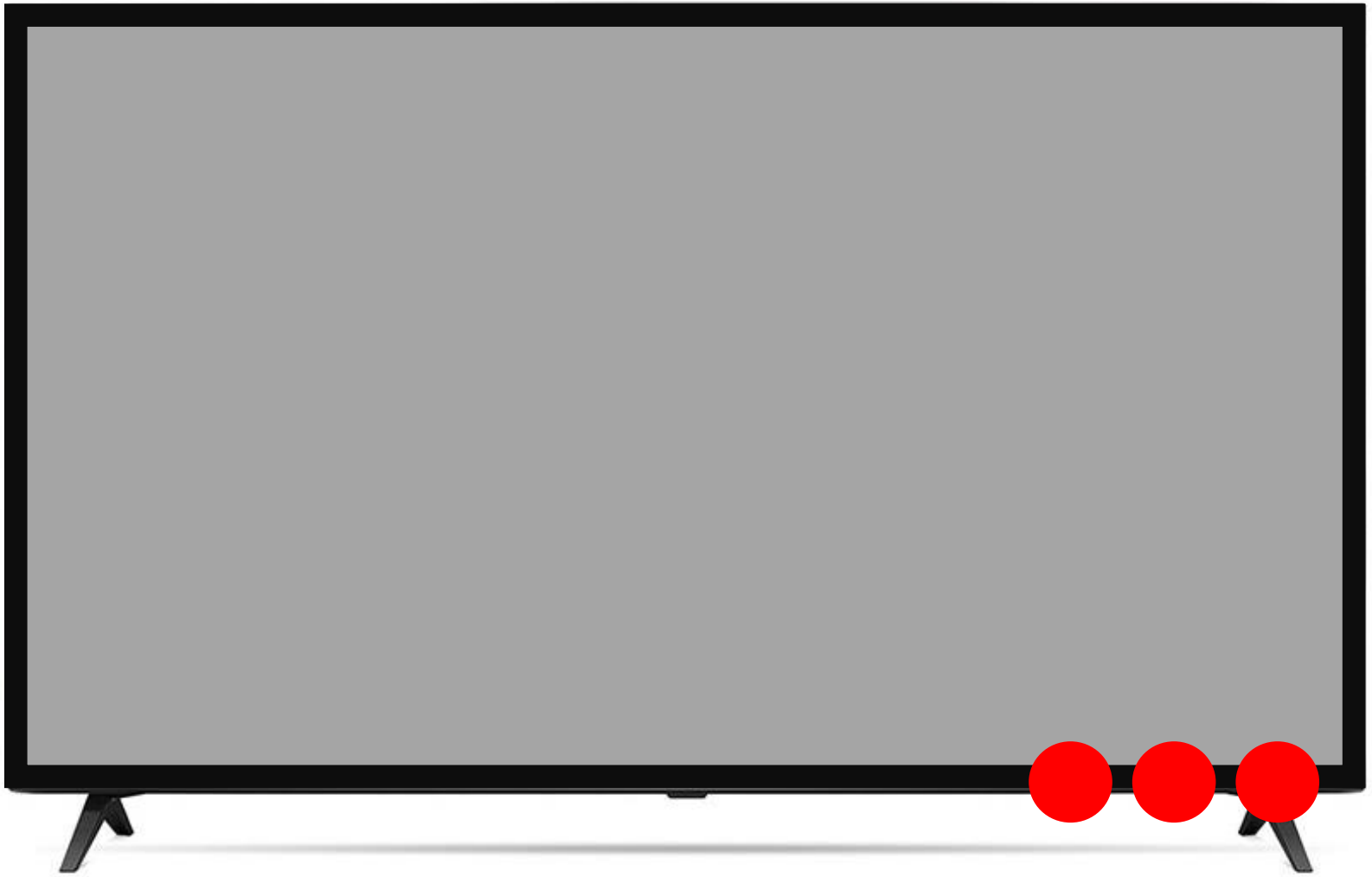
Encapsulation is the concept of **bundling data and methods** into a **class**, creating a capsule that **restricts direct access** to an object's data while providing **controlled access** through public methods, such as **getters** and **setters**

Abstraction

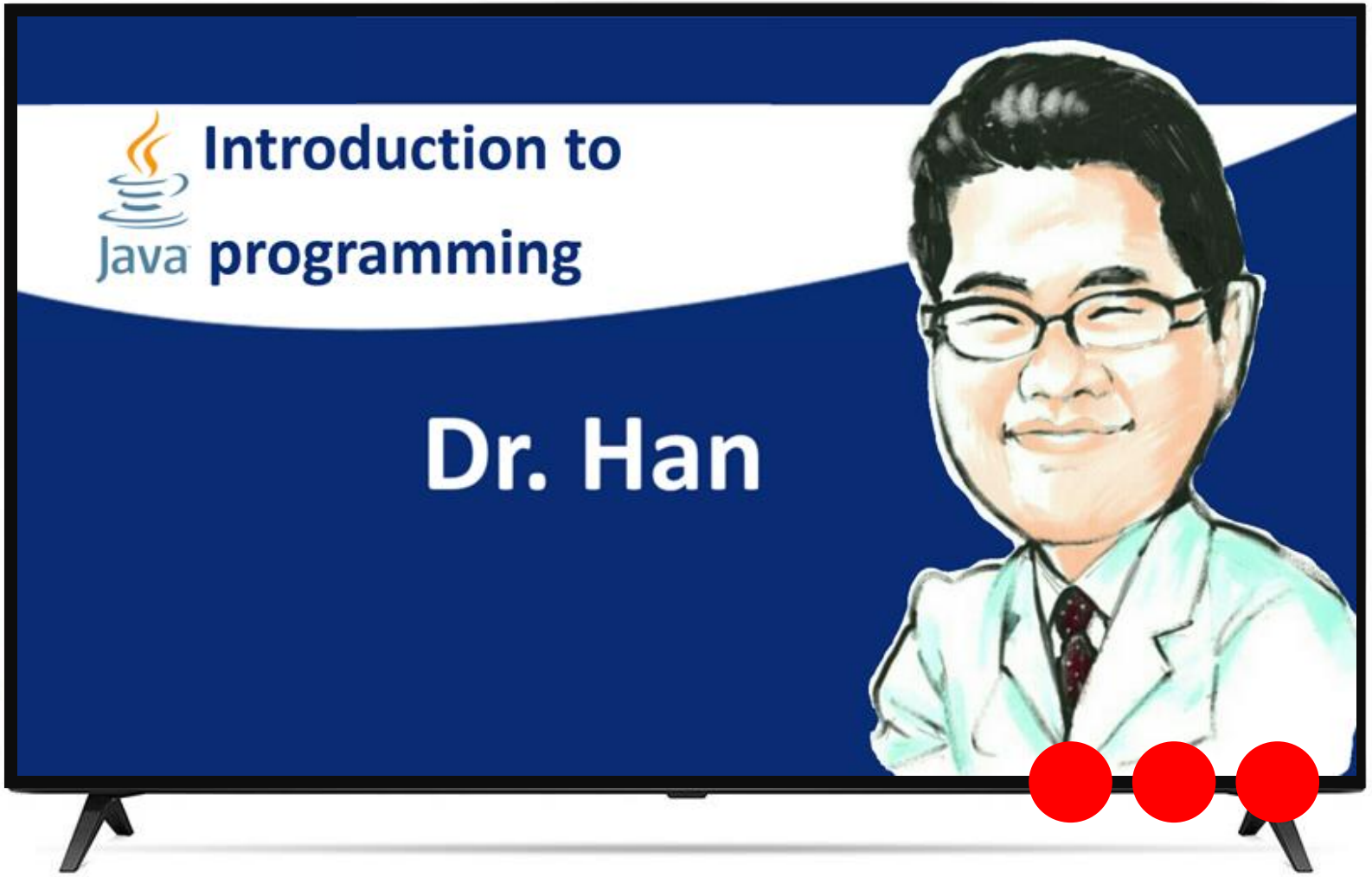
Abstraction



Abstraction

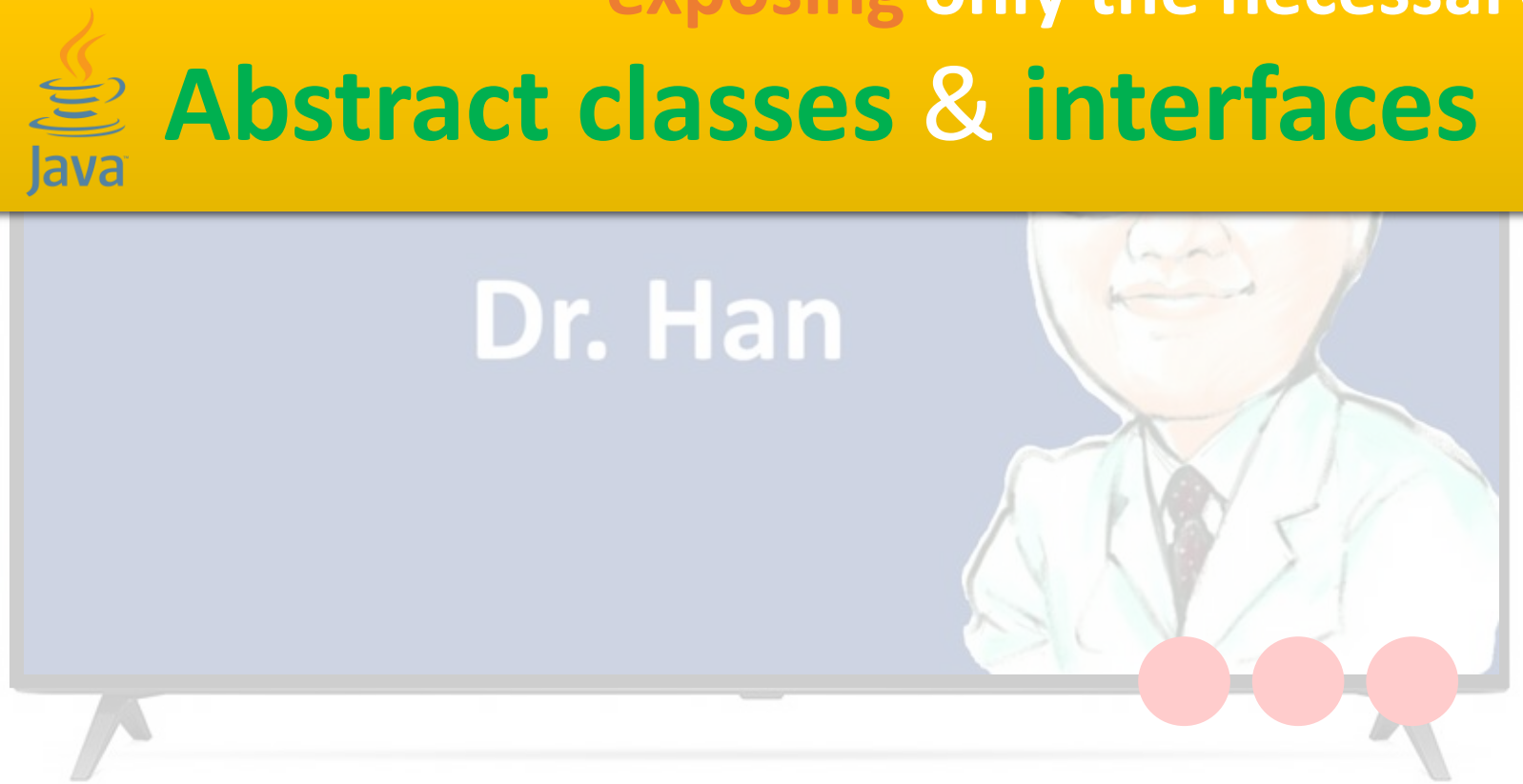


Abstraction



Abstraction1

Abstraction is about **hiding** **complex implementation details** and **exposing** **only the necessary functionality**.



Abstract classes & interfaces

Inheritance

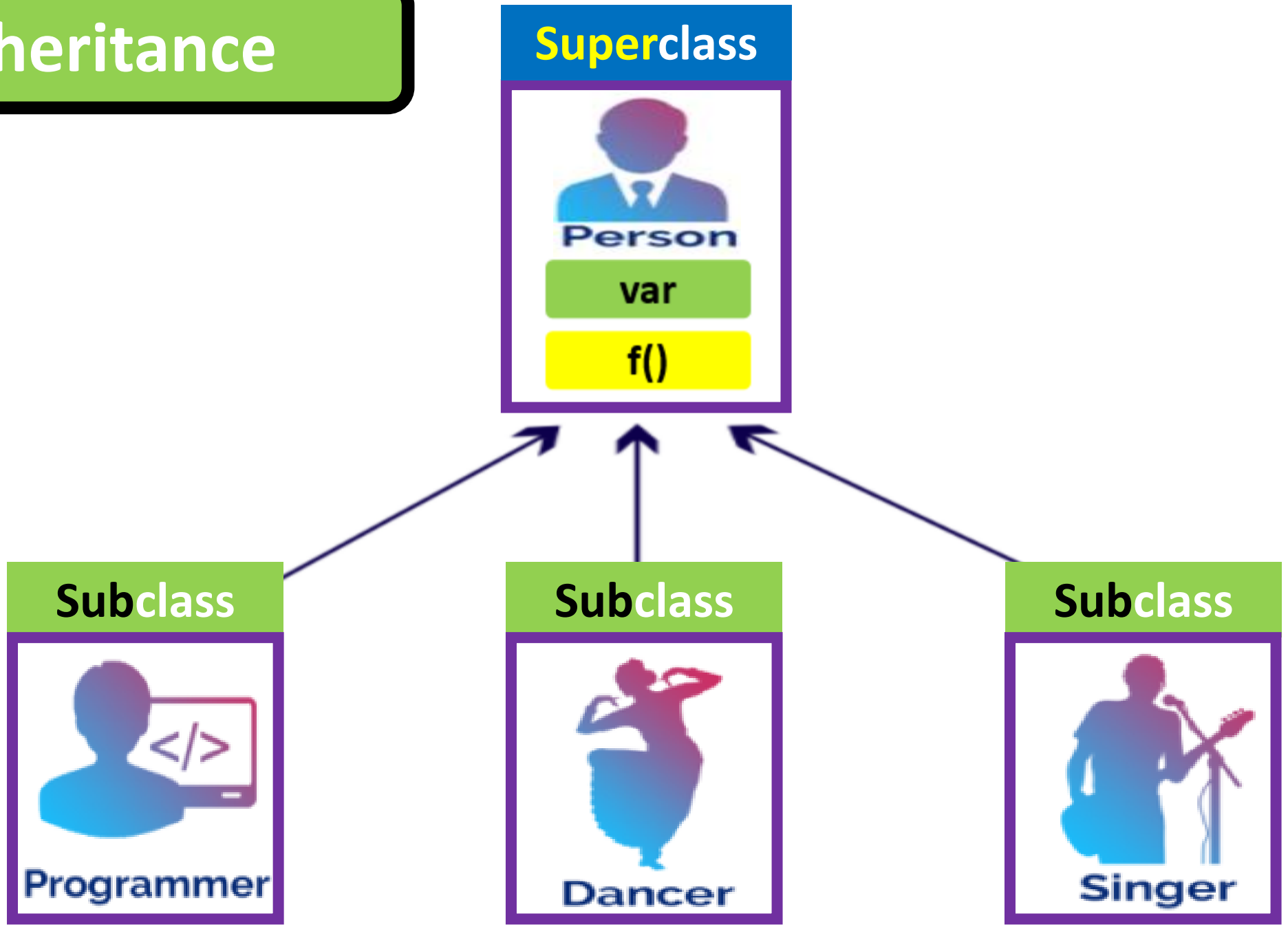
●●●

Inheritance

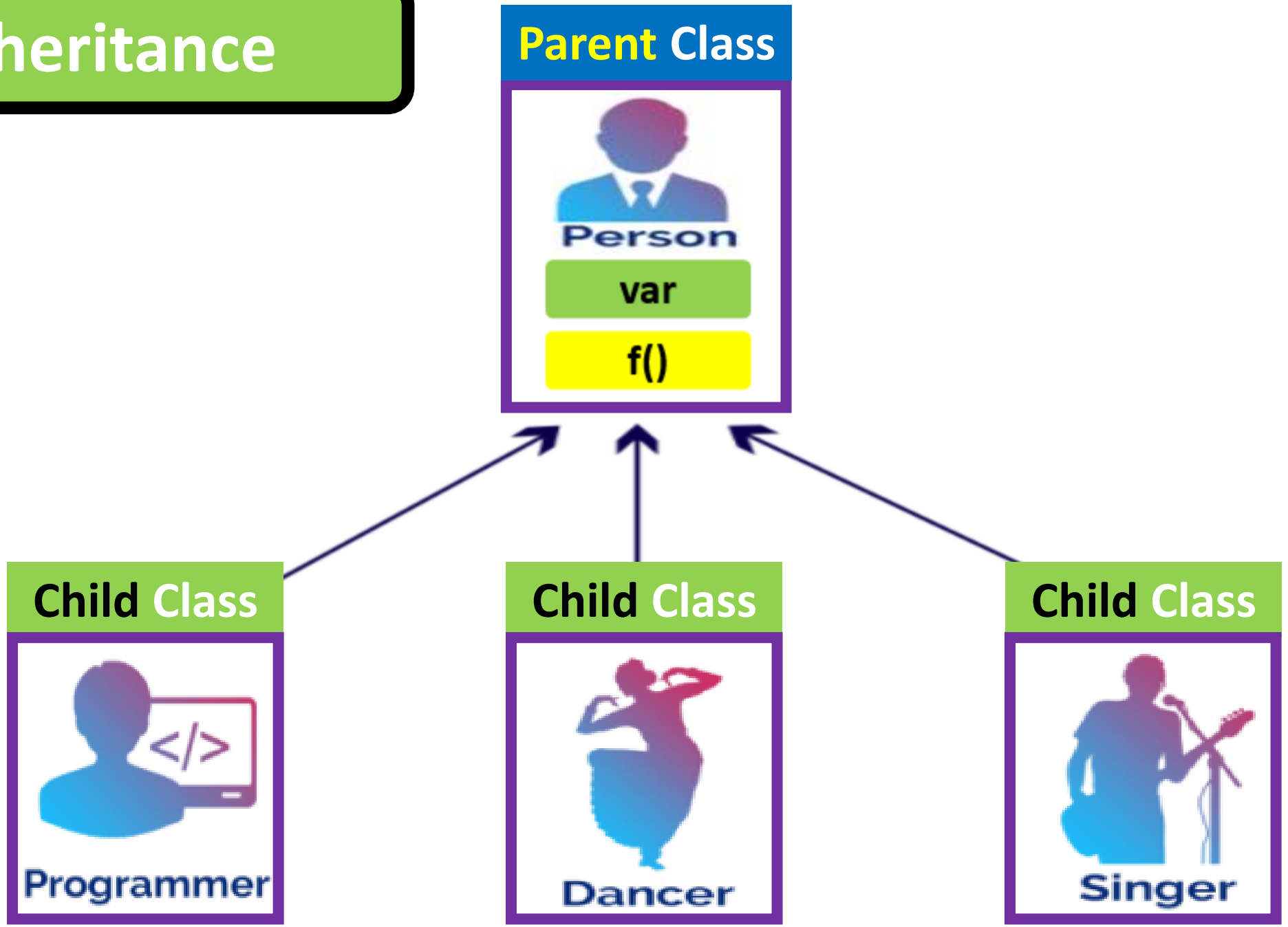
var

f()

Inheritance



Inheritance



Flying Dog

DOG
name breed age bark
say()

FlyingDOG
name breed age bark wing
say() fly()

DOG
name breed age bark
say()

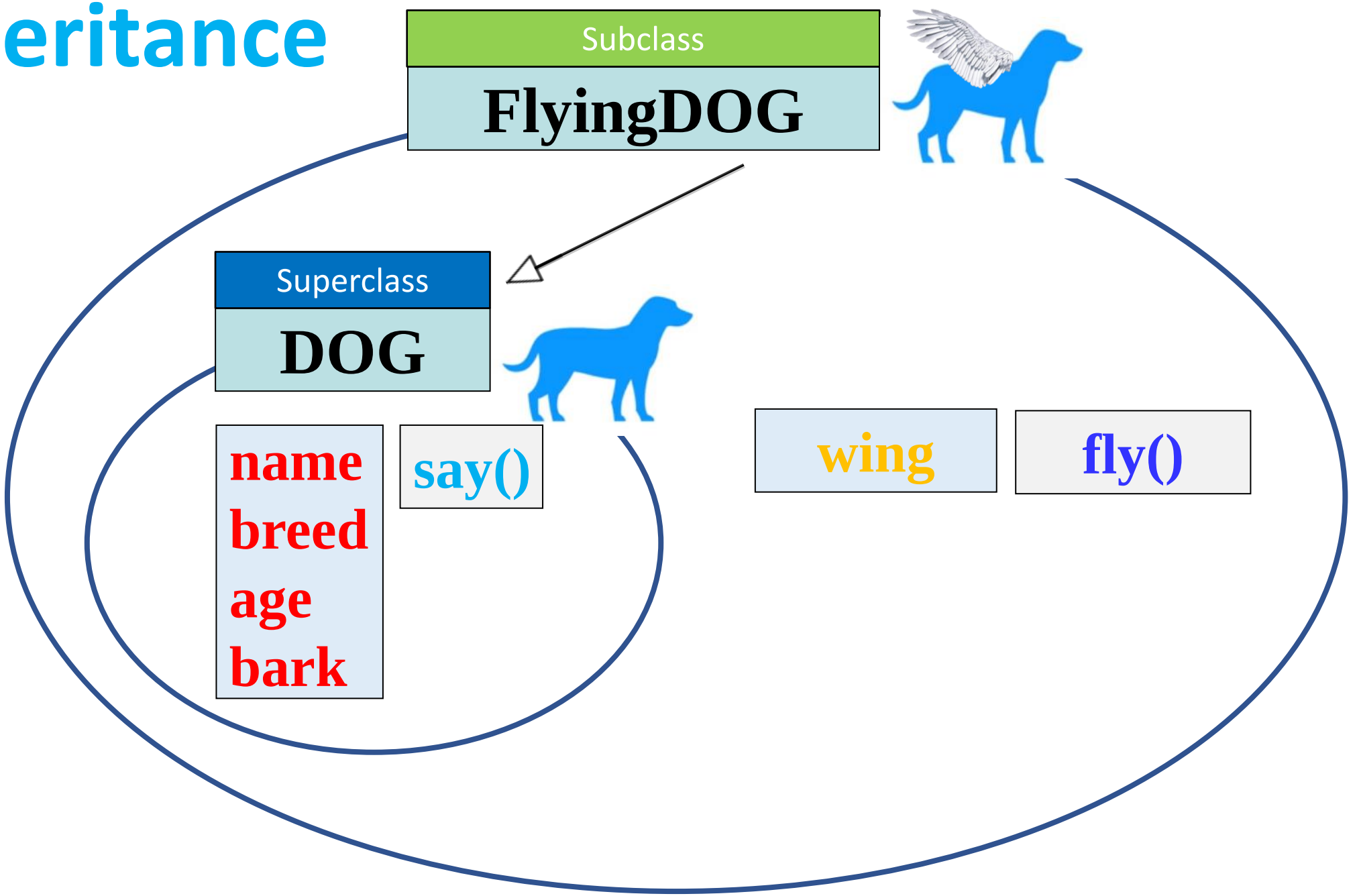
FlyingDOG
wing
fly()

Inheritance

Superclass
DOG
name breed age bark
say()

Subclass
FlyingDOG
wing
fly()

Inheritance



Inheritance

Subclass

FlyingDOG

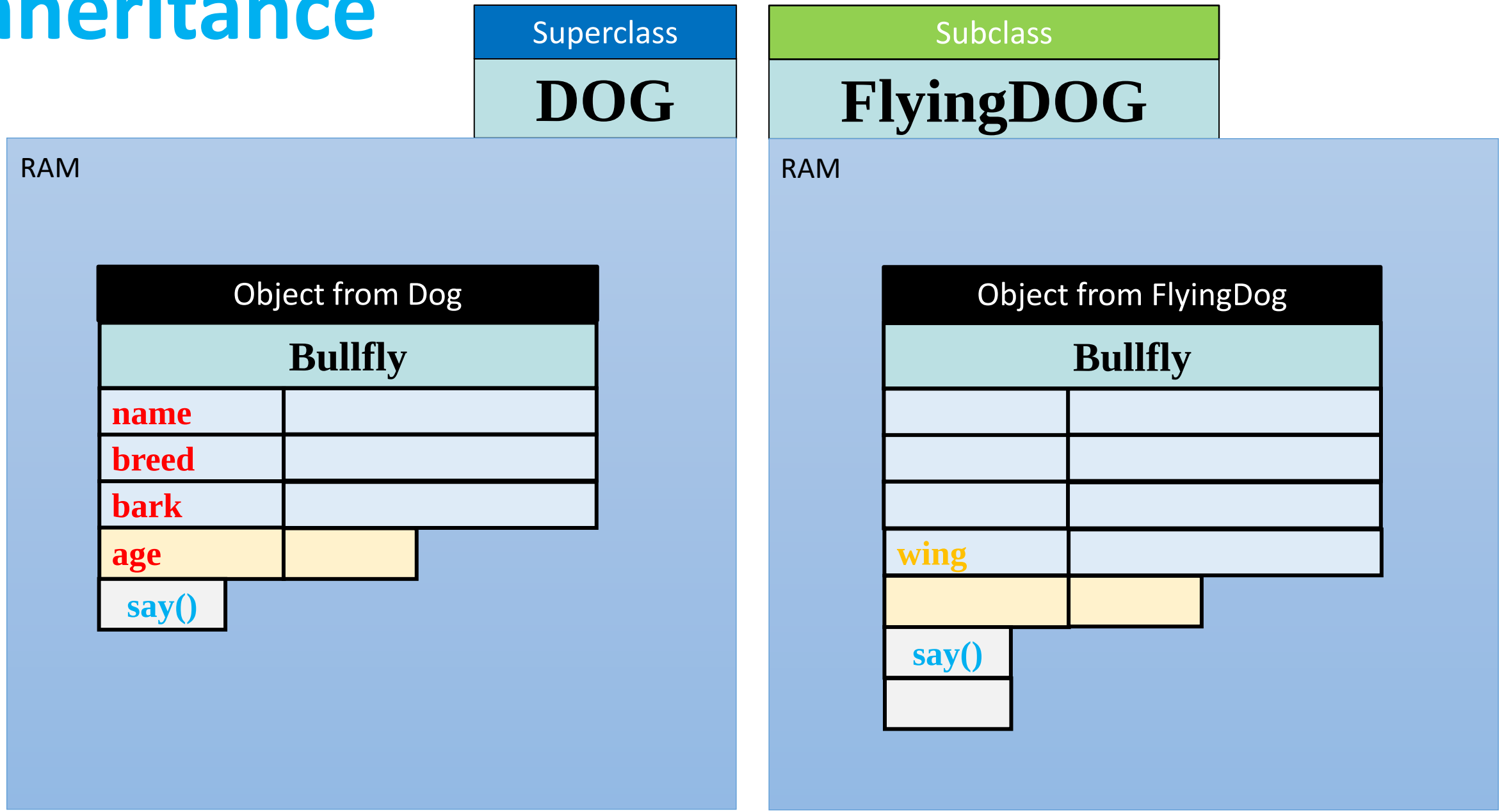
name
breed
age
bark

say()

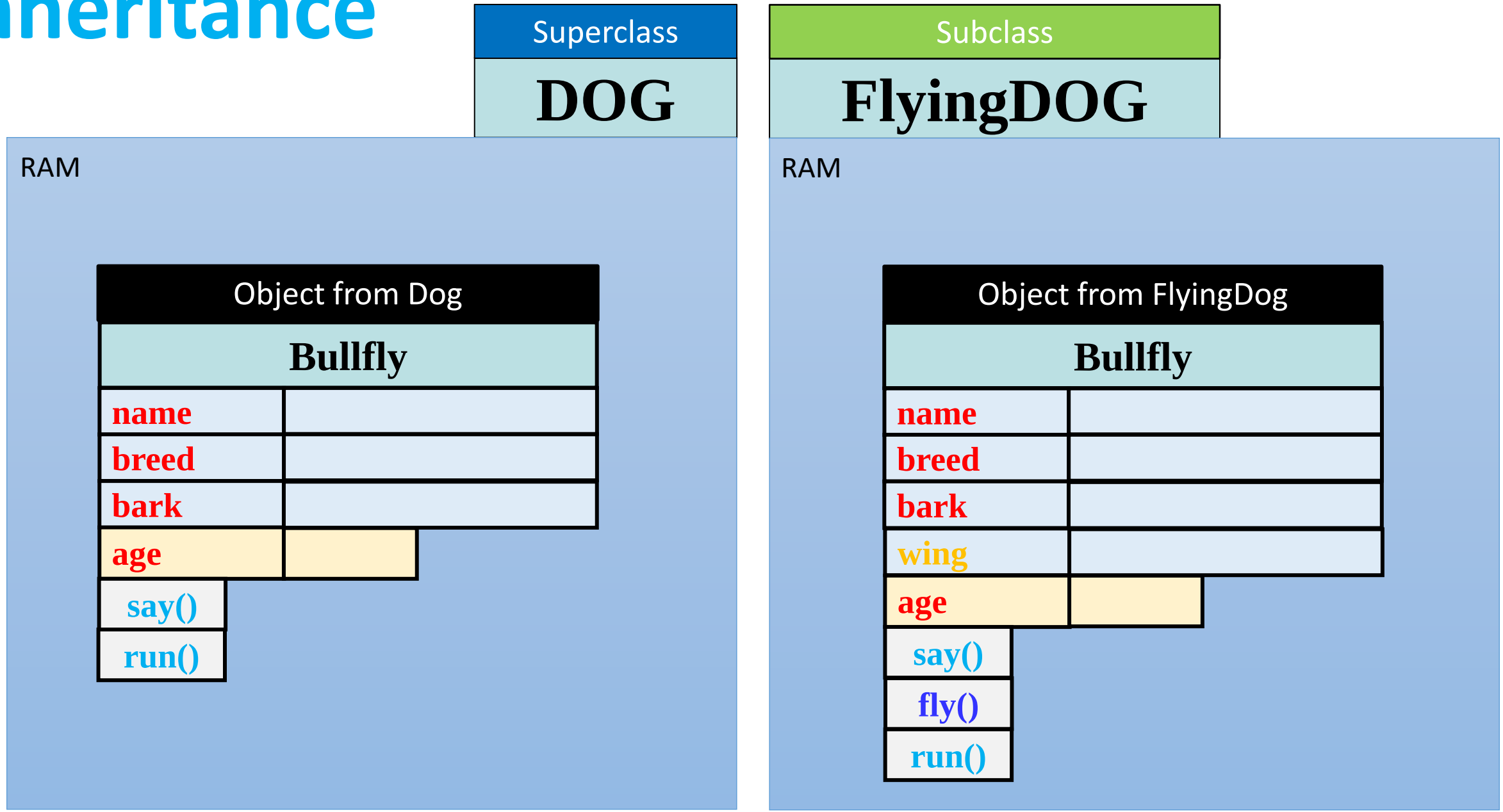
wing

fly()

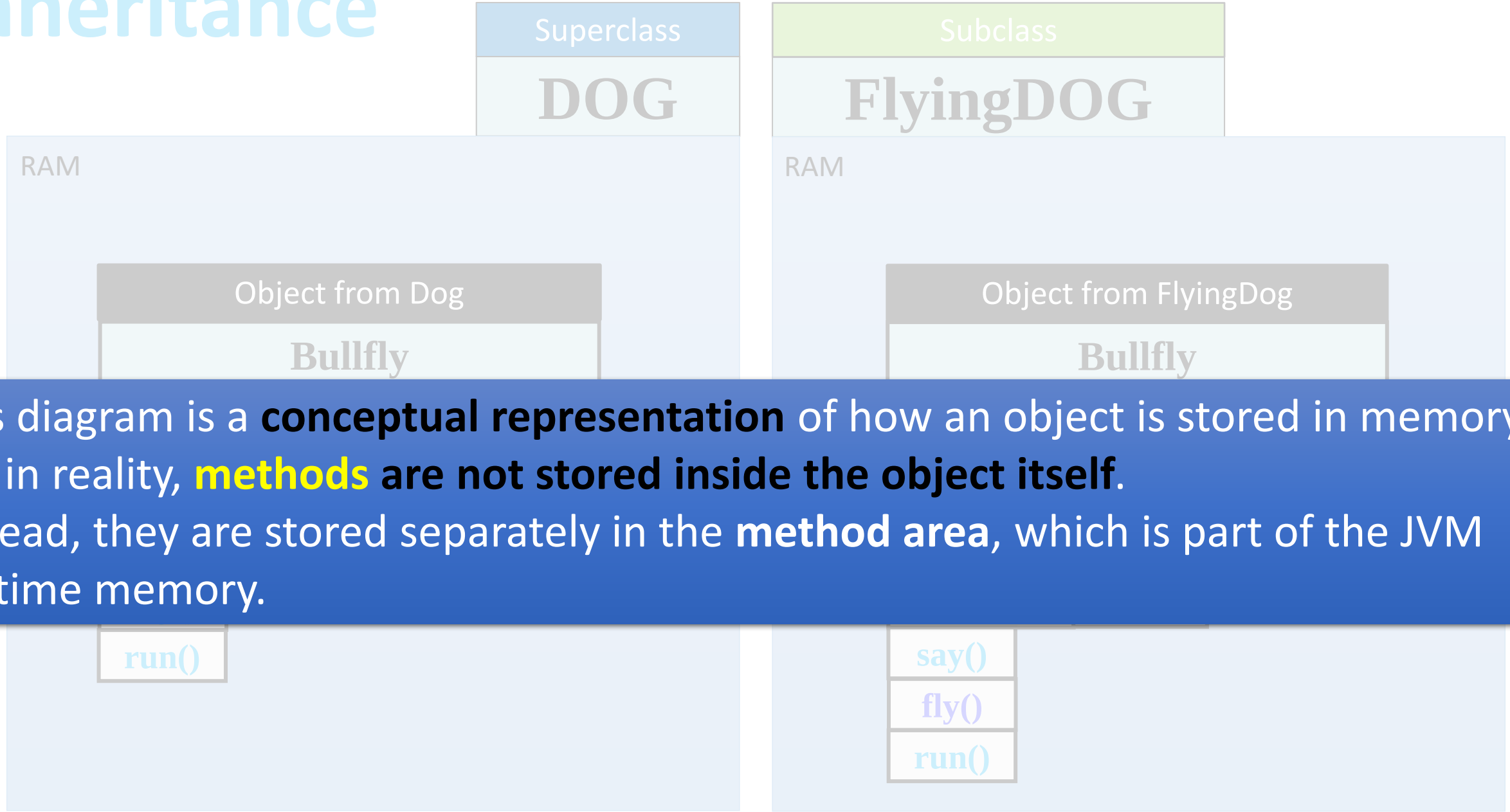
Inheritance



Inheritance



Inheritance



This diagram is a **conceptual representation** of how an object is stored in memory, but in reality, **methods are not stored inside the object itself**.
Instead, they are stored separately in the **method area**, which is part of the JVM runtime memory.

Superclass

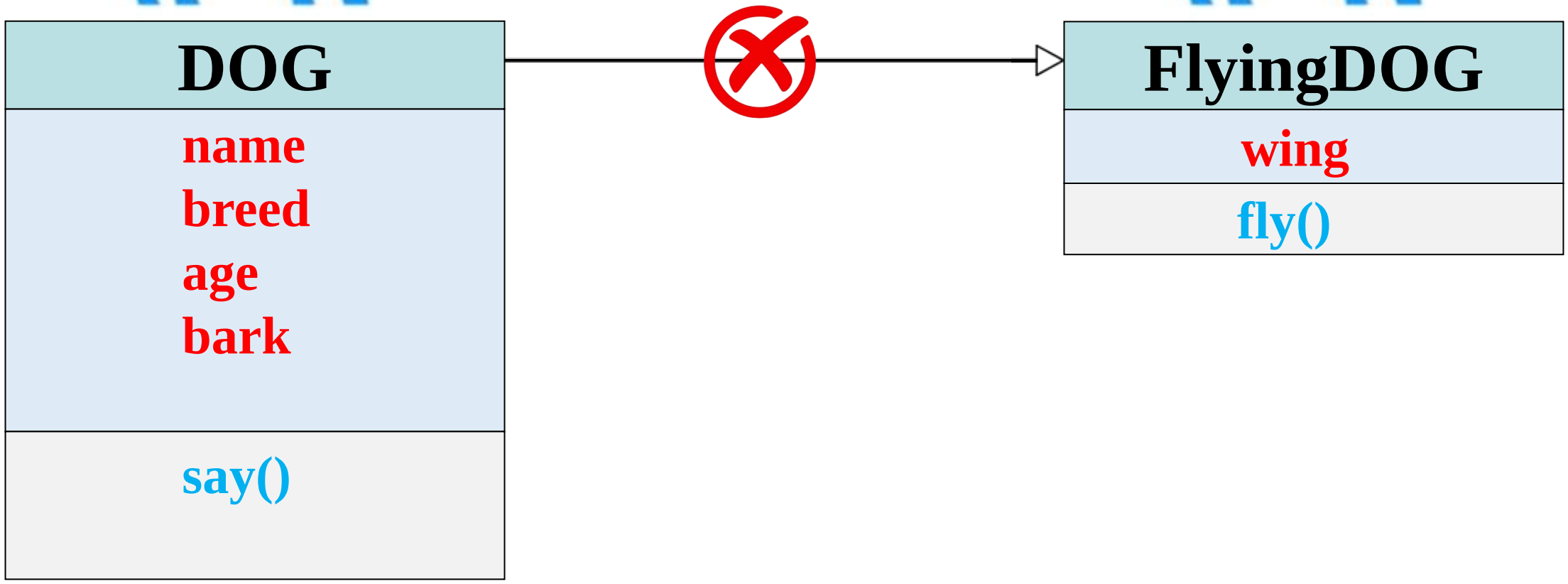
DOG
name breed age bark
say()

Subclass

FlyingDOG
wing
fly()

Superclass

Subclass



Superclass

Subclass

DOG
name breed age bark
say()

FlyingDOG
wing
fly()

Superclass

```
// Class Declaration
class Dog {
    // Properties
    String name;
    String breed;
    String bark;
    int age;

    // methods
    public void say() {
        System.out.println(name + " says: " + bark);
    }
}
```

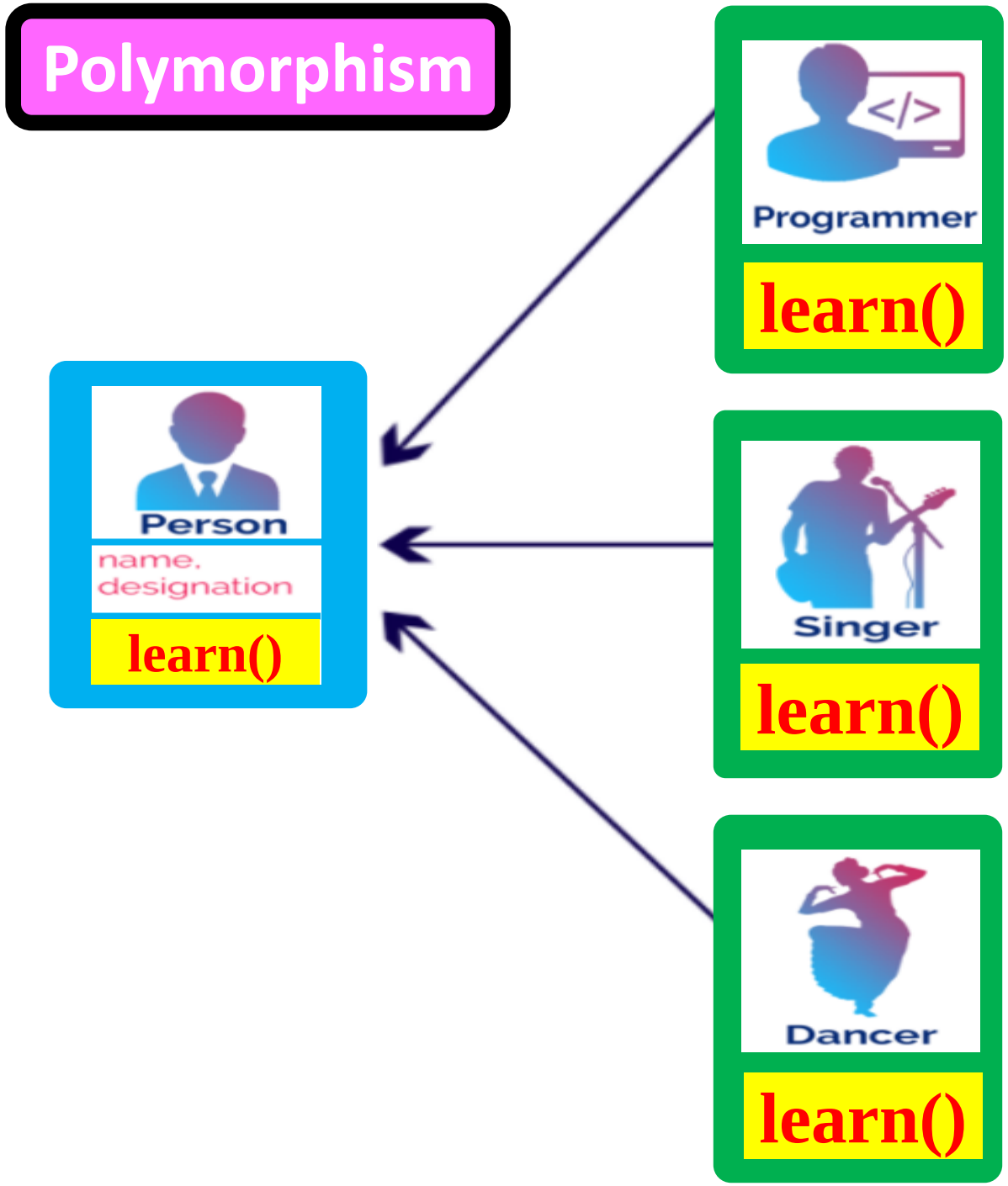
Subclass

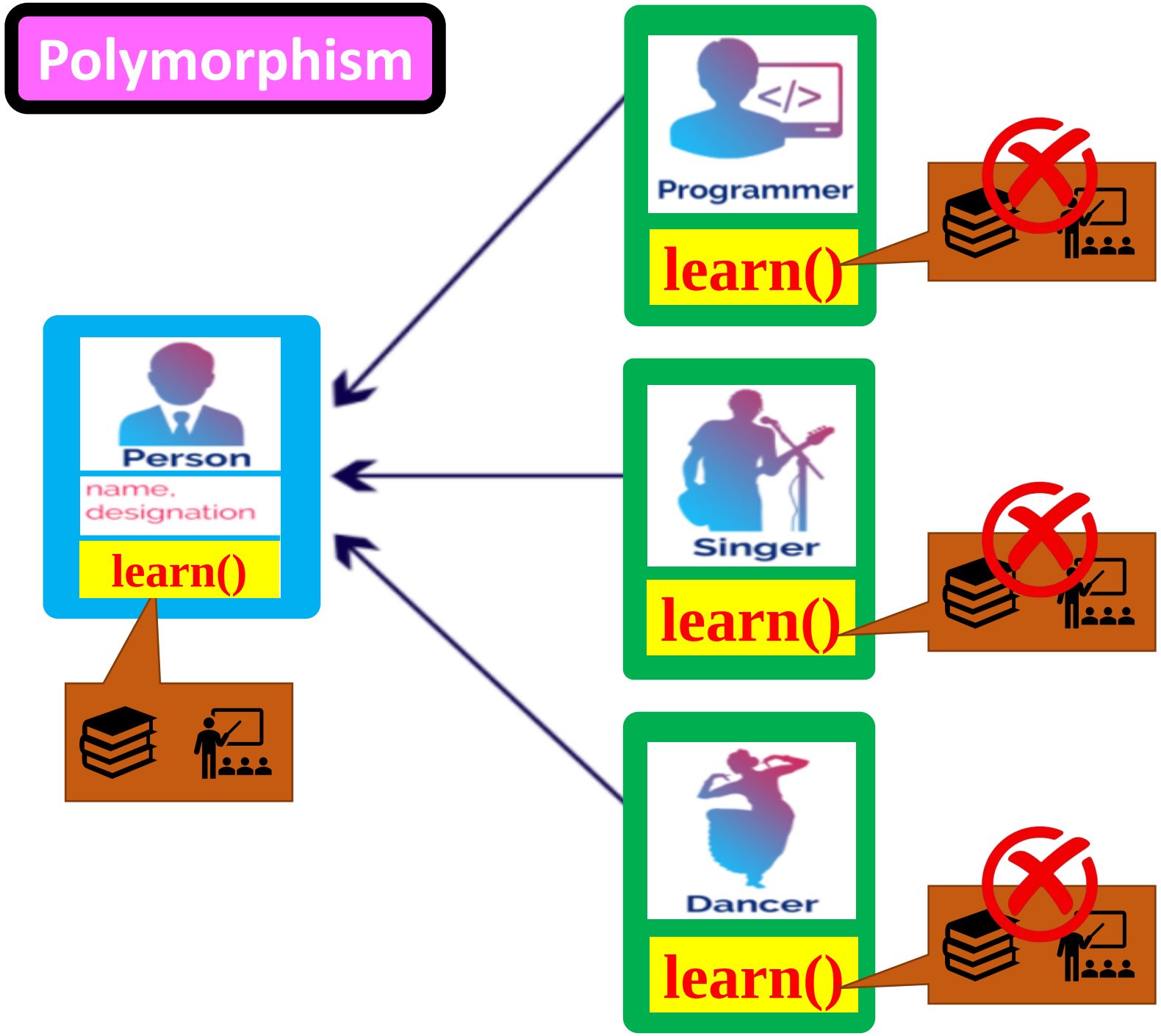
```
// Subclass Declaration
class FlyingDog extends Dog {
    // Properties
    String wing;

    // methods
    public void fly() {
        System.out.println(name + " flys!");
    }
}
```

Poly morphism

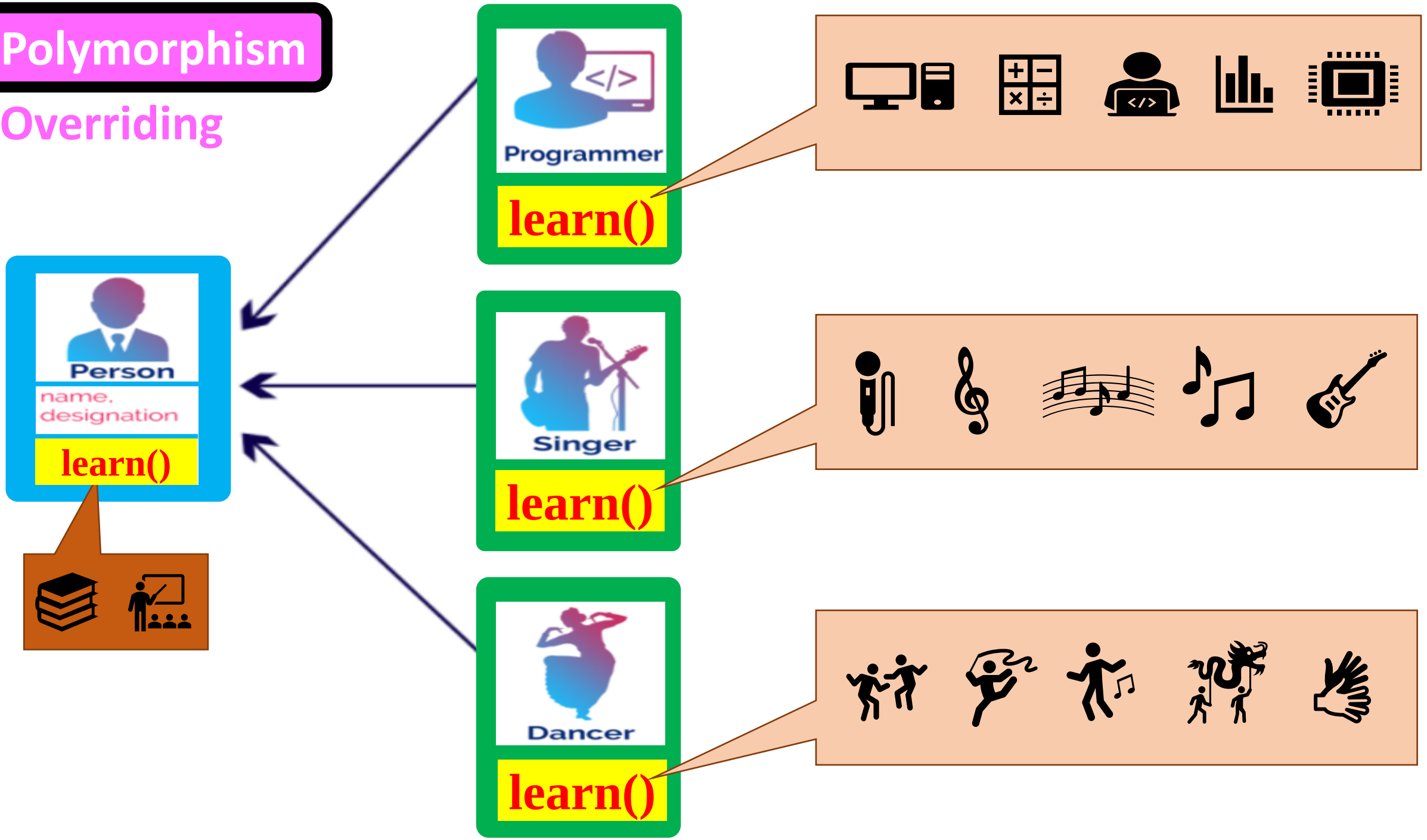
Many Form


Polymorphism
Person
name,
designation
learn()
Programmer
learn()
Singer
learn()
Dancer
learn()


Polymorphism
Programmer
learn()
Person
name,
designation
learn()
Singer
learn()
Dancer
learn()

Polymorphism

Overriding



Polymorphism

Overloading

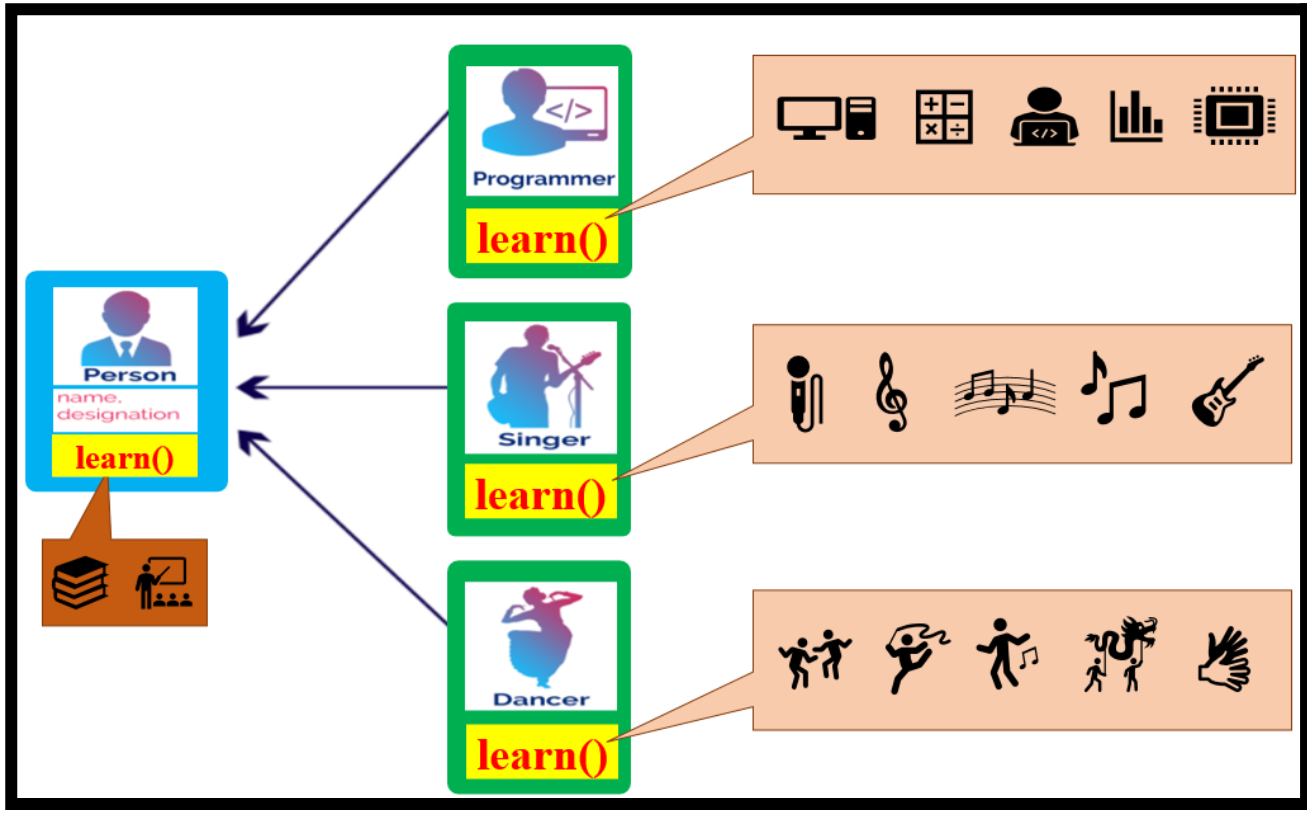
learn(fun, cpu, computer)

learn(music)

learn(party, event)

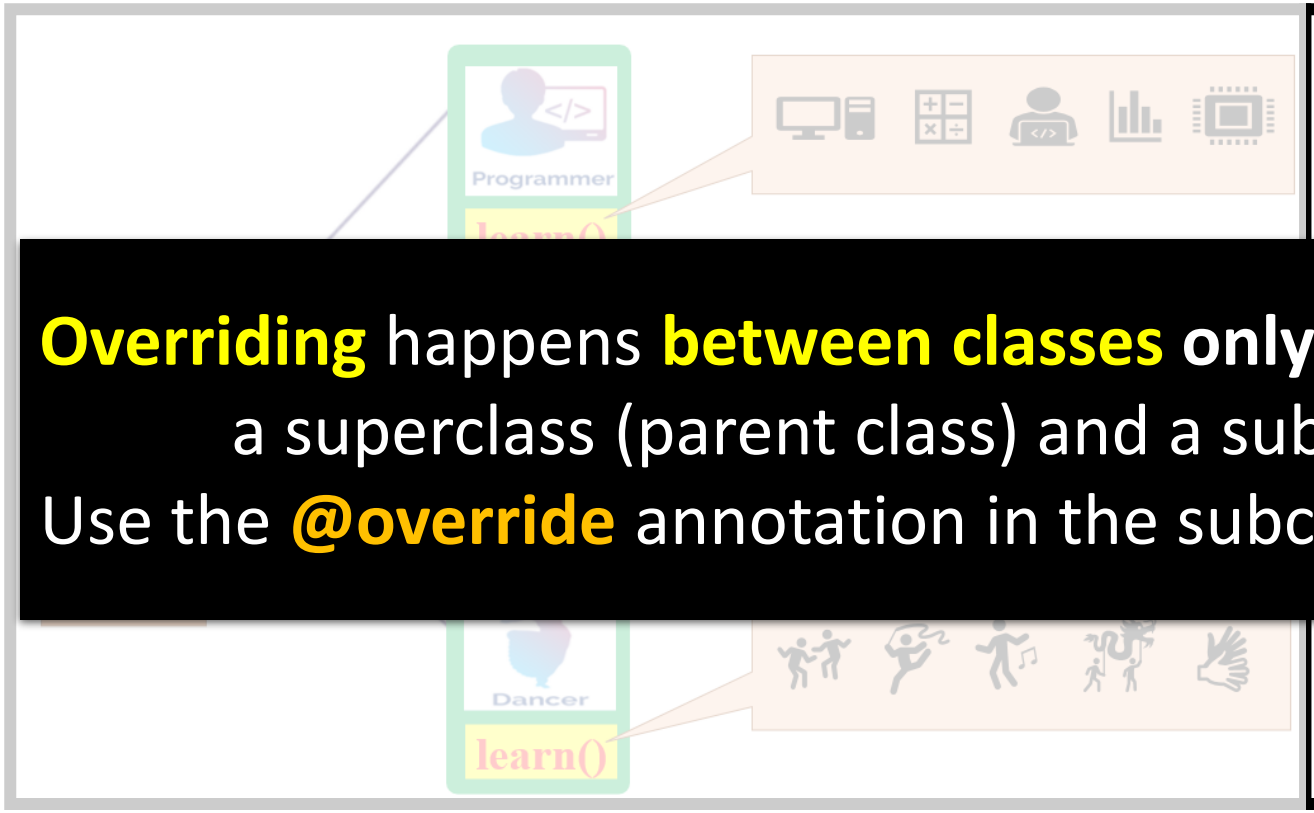
Polymorphism

overriding



Polymorphism

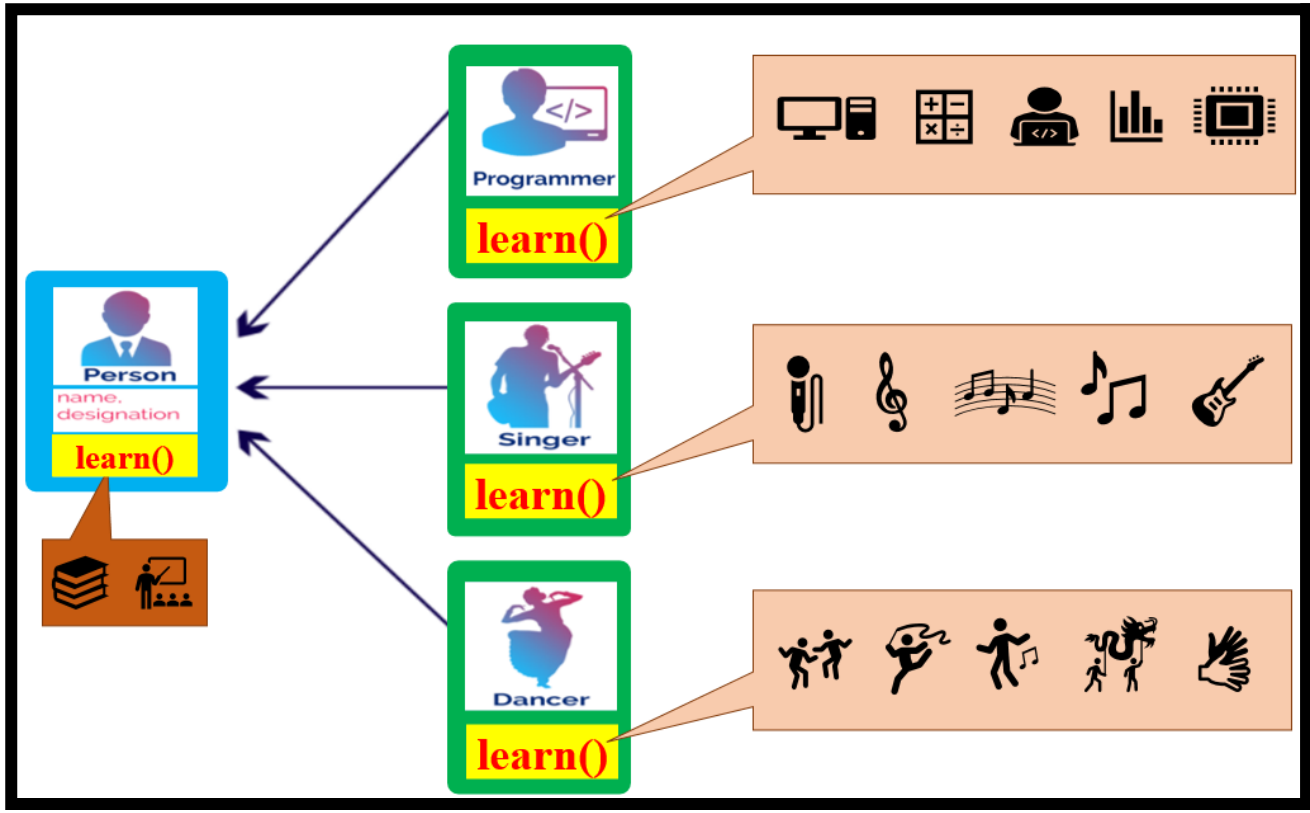
overriding



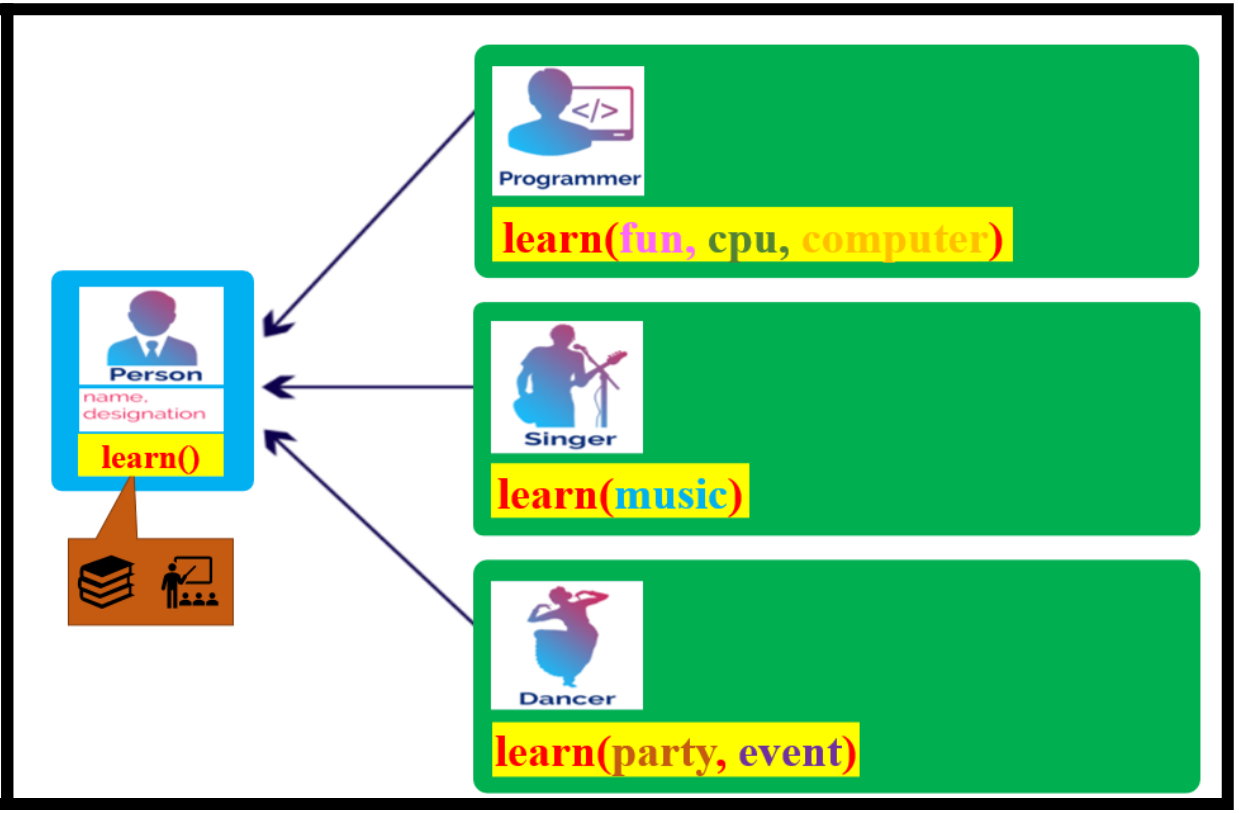
Overriding happens **between classes only**;
a superclass (parent class) and a subclass (child class) through inheritance.
Use the **@override** annotation in the subclass.

Polymorphism

overriding



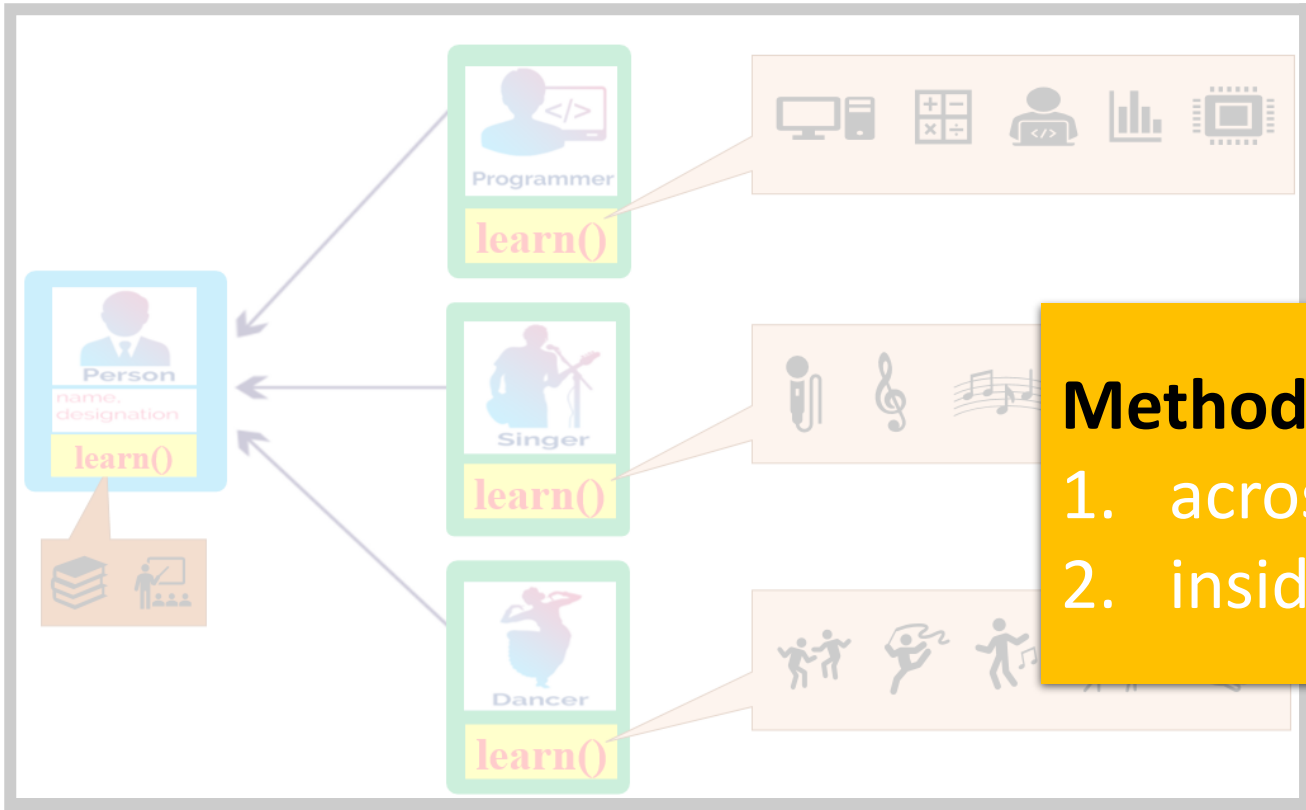
overloading

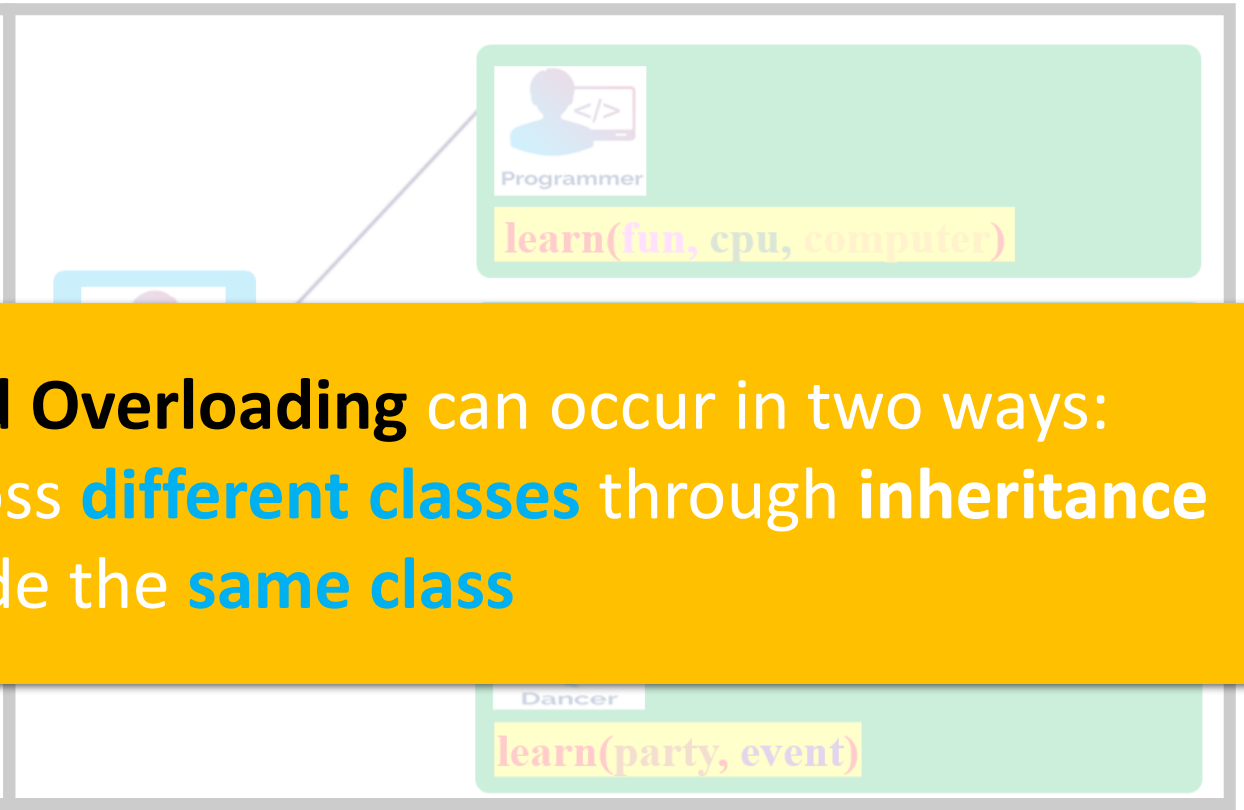


Polymorphism

overriding

overloading





Method Overloading can occur in two ways:

1. across **different classes** through **inheritance**
2. inside the **same class**

Superclass Method

run()

Superclass Method

run()

a black car is moving

Overriding:

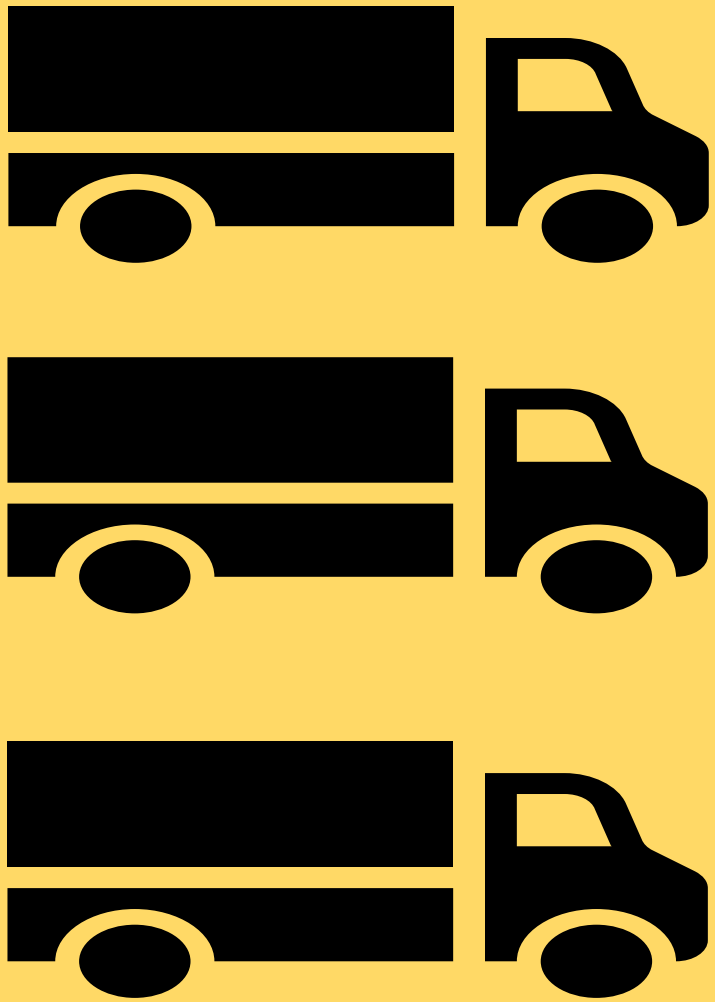
same name

but different implementation

run()

a pink truck is running

Base Method

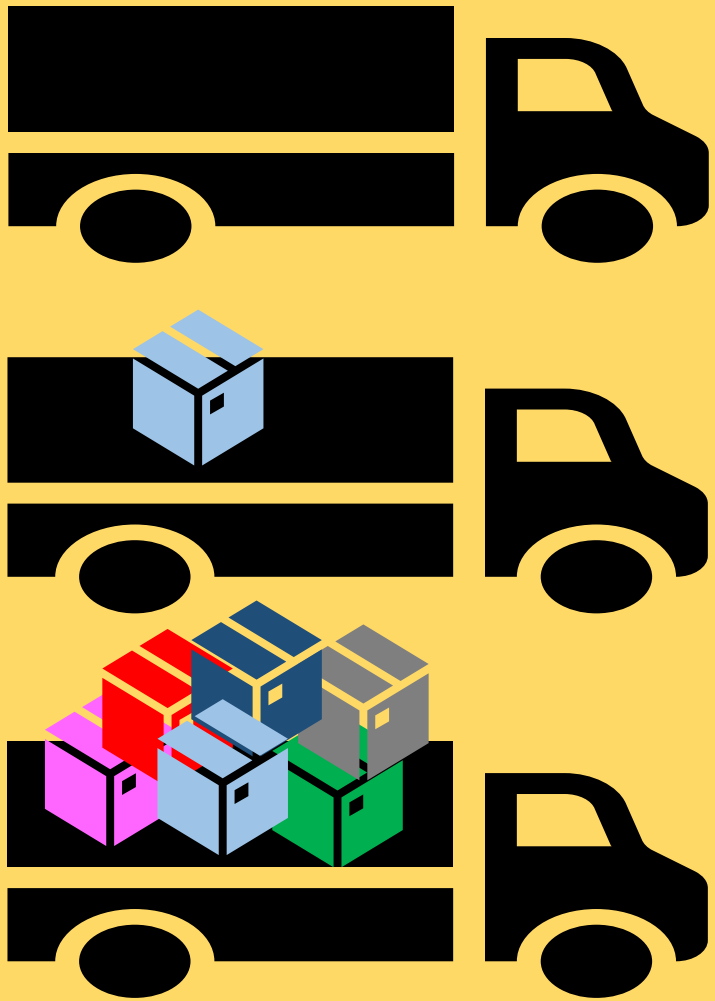


run()

Overloading:

same name

but different parameters



» run()

» run(a)

» run(a,b,c,d,e)

methods

```java
class A1 {
    public void m (double i) {
        System.out.println(i + ": A1");
    }
}

class A2 extends A1 {
    @Override
    public void m (double i) {
        System.out.println(i + ": A2");
    }
}

public class Overriding {
    public static void main(String[] args) {

        A2 a = new A2();
        a.m(1);
        a.m(1.0);
    }
}
```

```java
class A1 {
    public void m (double i) {
        System.out.println(i + ": A1");
    }
}

class A2 extends A1 {
    //Overload
    public void m (int i) {
        System.out.println(i + ": A2");
    }
}

public class Overloading {
    public static void main(String[] args) {

        A2 a = new A2();
        a.m(1);
        a.m(1.0);
    }
}
```

methods

same name, same parameters

```
1  class A1 {
2      public void m (double i) {
3          System.out.println(i + ": A1");
4      }
5  }
6
7  class A2 extends A1 {
8      @Override
9      public void m (double i) {
10         System.out.println(i + ": A2");
11     }
12 }
13
14 public class Overriding {
15     public static void main(String[] args) {
16
17         A2 a = new A2();
18         a.m(1);
19         a.m(1.0);
20     }
21 }
```

```
25 class A1 {
26     public void m (double i) {
27         System.out.println(i + ": A1");
28     }
29 }
30
31 class A2 extends A1 {
32     //Overload
33     public void m (int i) {
34         System.out.println(i + ": A2");
35     }
36 }
37
38 public class Overloading {
39     public static void main(String[] args) {
40
41         A2 a = new A2();
42         a.m(1);
43         a.m(1.0);
44     }
45 }
```

methods overriding

same name, same parameters

different implementation

```
1  class A1 {
2      public void m (double i) {
3          System.out.println(i + ": A1");
4      }
5  }
6
7  class A2 extends A1 {
8      @Override
9      public void m (double i) {
10         System.out.println(i + ": A2");
11     }
12 }
13
14 public class Overriding {
15     public static void main(String[] args) {
16
17         A2 a = new A2();
18         a.m(1);
19         a.m(1.0);
20     }
21 }
```

```
25 class A1 {
26     public void m (double i) {
27         System.out.println(i + ": A1");
28     }
29 }
30
31 class A2 extends A1 {
32     //Overload
33     public void m (int i) {
34         System.out.println(i + ": A2");
35     }
36 }
37
38 public class Overloading {
39     public static void main(String[] args) {
40
41         A2 a = new A2();
42         a.m(1);
43         a.m(1.0);
44     }
45 }
```

methods

overloading

same name

different parameters

```
1  class A1 {
2      public void m (double i) {
3          System.out.println(i + ": A1");
4      }
5  }
6
7  class A2 extends A1 {
8      @Override
9      public void m (double i) {
10         System.out.println(i + ": A2");
11     }
12 }
13
14 public class Overriding {
15     public static void main(String[] args) {
16
17         A2 a = new A2();
18         a.m(1);
19         a.m(1.0);
20     }
21 }
```

```
25 class A1 {
26     public void m (double i) {
27         System.out.println(i + ": A1");
28     }
29 }
30
31 class A2 extends A1 {
32     //Overload
33     public void m (int i) {
34         System.out.println(i + ": A2");
35     }
36 }
37
38 public class Overloading {
39     public static void main(String[] args) {
40
41         A2 a = new A2();
42         a.m(1);
43         a.m(1.0);
44     }
45 }
```

methods overriding vs. overloading

```java
class A1 {
    public void m (double i) {
        System.out.println(i + ": A1");
    }
}

class A2 extends A1 {
    @Override
    public void m (double i) {
        System.out.println(i + ": A2");
    }
}

public class Overriding {
    public static void main(String[] args) {

        A2 a = new A2();
        a.m(1);
        a.m(1.0);
    }
}
```

```java
class A1 {
    public void m (double i) {
        System.out.println(i + ": A1");
    }
}

class A2 extends A1 {
    //Overload
    public void m (int i) {
        System.out.println(i + ": A2");
    }
}

public class Overloading {
    public static void main(String[] args) {

        A2 a = new A2();
        a.m(1);
        a.m(1.0);
    }
}
```

methods overriding vs. overloading

```java
class A1 {
    public void m (double i) {
        System.out.println(i + ": A1");
    }
}

class A2 extends A1 {
    @Override
    public void m (double i) {
        System.out.println(i + ": A2");
    }
}

public class Overriding {
    public static void main(String[] args) {

        A2 a = new A2();
        a.m(1); 1
        a.m(1.0);
    }
}
```

```java
class A1 {
    public void m (double i) {
        System.out.println(i + ": A1");
    }
}

class A2 extends A1 {
    //Overload
    public void m (int i) {
        System.out.println(i + ": A2");
    }
}

public class Overloading {
    public static void main(String[] args) {

        A2 a = new A2();
        a.m(1);
        a.m(1.0);
    }
}
```

methods overriding vs. overloading

```
1  class A1 {
2      public void m (double i) {
3          System.out.println(i + ": A1");
4      }
5  }
6
7  class A2 extends A1 {
8      @Override
9      public void m (double i) {
10         System.out.println(i + ": A2");
11     }
12 }
13
14 public class Overriding {
15     public static void main(String[] args) {
16
17         A2 a = new A2();
18         a.m(1);
19         a.m(1.0);
20 1.0: A2
21 }
```

2 3 4 1

```
25 class A1 {
26     public void m (double i) {
27         System.out.println(i + ": A1");
28     }
29 }
30
31 class A2 extends A1 {
32     //Overload
33     public void m (int i) {
34         System.out.println(i + ": A2");
35     }
36 }
37
38 public class Overloading {
39     public static void main(String[] args) {
40
41         A2 a = new A2();
42         a.m(1);
43         a.m(1.0);
44     }
45 }
```

methods overriding vs. overloading

```
1  class A1 {
2      public void m (double i) {
3          System.out.println(i + ": A1");
4      }
5  }
6
7  class A2 extends A1 {
8      @Override
9      public void m (double i) {
10         System.out.println(i + ": A2");
11     }
12 }
13
14 public class Overriding {
15     public static void main(String[] args) {
16
17         A2 a = new A2();
18         a.m(1);
19         a.m(1.0);
20 1.0: A2
21
```

2 3 4 1 5

```
25 class A1 {
26     public void m (double i) {
27         System.out.println(i + ": A1");
28     }
29 }
30
31 class A2 extends A1 {
32     //Overload
33     public void m (int i) {
34         System.out.println(i + ": A2");
35     }
36 }
37
38 public class Overloading {
39     public static void main(String[] args) {
40
41         A2 a = new A2();
42         a.m(1);
43         a.m(1.0);
44     }
45 }
```

methods overriding vs. overloading

```
1  class A1 {
2      public void m (double i) {
3          System.out.println(i + ": A1");
4      }
5  }
6
7  class A2 extends A1 {
8      @Override
9      public void m (double i) {
10         System.out.println(i + ": A2");
11     }
12 }
13
14 public class Overriding {
15     public static void main(String[] args) {
16
17         A2 a = new A2();
18         a.m(1);
19         a.m(1.0);
20
21 }
```

```
1.0: A2
1.0: A2
```

```
25 class A1 {
26     public void m (double i) {
27         System.out.println(i + ": A1");
28     }
29 }
30
31 class A2 extends A1 {
32     //Overload
33     public void m (int i) {
34         System.out.println(i + ": A2");
35     }
36 }
37
38 public class Overloading {
39     public static void main(String[] args) {
40
41         A2 a = new A2();
42         a.m(1);
43         a.m(1.0);
44     }
45 }
```

methods overriding vs. overloading

```java
class A1 {
    public void m (double i) {
        System.out.println(i + ": A1");
    }
}

class A2 extends A1 {
    @Override
    public void m (double i) {
        System.out.println(i + ": A2");
    }
}

public class Overriding {
    public static void main(String[] args) {

        A2 a = new A2();
        a.m(1);
        a.m(1.0);
    }
}
```

```java
class A1 {
    public void m (double i) {
        System.out.println(i + ": A1");
    }
}

class A2 extends A1 {
    //Overload
    public void m (int i) {
        System.out.println(i + ": A2");
    }
}

public class Overloading {
    public static void main(String[] args) {

        A2 a = new A2();
        a.m(1);
        a.m(1.0);
    }
}
```

methods overriding vs. overloading

```java
class A1 {
    public void m (double i) {
        System.out.println(i + ": A1");
    }
}

class A2 extends A1 {
    @Override
    public void m (double i) {
        System.out.println(i + ": A2");
    }
}

public class Overriding {
    public static void main(String[] args) {

        A2 a = new A2();
        a.m(1);
        a.m(1.0);
    }
}
```

```java
class A1 {
    public void m (double i) {
        System.out.println(i + ": A1");
    }
}

class A2 extends A1 {
    //Overload
    public void m (int i) {
        System.out.println(i + ": A2");
    }
}

public class Overloading {
    public static void main(String[] args) {

        A2 a = new A2();
        a.m(1);
        a.m(1.0);
```

```
1: A2
```

methods overriding vs. overloading

```java
class A1 {
    public void m (double i) {
        System.out.println(i + ": A1");
    }
}

class A2 extends A1 {
    @Override
    public void m (double i) {
        System.out.println(i + ": A2");
    }
}

public class Overriding {
    public static void main(String[] args) {

        A2 a = new A2();
        a.m(1);
        a.m(1.0);
    }
}
```

```java
class A1 {
    public void m (double i) {
        System.out.println(i + ": A1");
    }
}

class A2 extends A1 {
    //Overload
    public void m (int i) {
        System.out.println(i + ": A2");
    }
}

public class Overloading {
    public static void main(String[] args) {

        A2 a = new A2();
        a.m(1);
        a.m(1.0);
```

```
1: A2
```

methods overriding vs. overloading

```java
class A1 {
    public void m (double i) {
        System.out.println(i + ": A1");
    }
}

class A2 extends A1 {
    @Override
    public void m (double i) {
        System.out.println(i + ": A2");
    }
}

public class Overriding {
    public static void main(String[] args) {

        A2 a = new A2();
        a.m(1);
        a.m(1.0);
    }
}
```

```java
class A1 {                          6
    public void m (double i) {
        System.out.println(i + ": A1");  7
    }
}

                                    2
class A2 extends A1 {
    //Overload
                                    3
    public void m (int i) {
                                    4
        System.out.println(i + ": A2");
    }
}

public class Overloading {
    public static void main(String[] args) {

        A2 a = new A2();
        a.m(1);  1
                                    5
        a.m(1.0);
```

```
1: A2
1.0: A1
```

Overloading:

same name

different parameters

```
1  class Parent {
2      void md(int a) {
3          System.out.println("md-Parent: " + a);
4      }
5  }
6
7  class Child extends Parent {
8      void md(int a, int b) { // Overloading in the subclass
9          System.out.println("md-Child: " + a + " and " + b);
10     }
11 }
12
13 public class overloading1 {
14     public static void main(String[] args) {
15         Child obj = new Child();
16         obj.md(10);
17         obj.md(10, 20);
18     }
19 }
```

Overloading:

same name

different parameters

```
1  class Parent {
2      void md(int a) {
3          System.out.println("md-Parent: " + a);
4      }
5  }
6
7  class Child extends Parent {
8      void md(int a, int b) { // Overloading in the subclass
9          System.out.println("md-Child: " + a + " and " + b);
10     }
11 }
12
13 public class overloading1 {
14     public static void main(String[] args) {
15         Child obj = new Child();
16         obj.md(10);
17         obj.md(10, 20);
18     }
19 }
```

Overloading:

same name

different parameters

```
1  class Parent {
2      void md(int a) {
3          System.out.println("md-Parent: " + a);
4      }
5  }
6
7  class Child extends Parent {
8      void md(int a, int b) { // Overloading in the subclass
9          System.out.println("md-Child: " + a + " and " + b);
10     }
11 }
12
13 public class overloading1 {
14     public static void main(String[] args) {
15         Child obj = new Child();
16         obj.md(10);
17         obj.md(10, 20);
18     }
19 }
```

```
md-Parent: 10
```

Overloading:

same name

different parameters

```
1  class Parent {
2      void md(int a) {
3          System.out.println("md-Parent: " + a);
4      }
5  }
6
7  class Child extends Parent {
8      void md(int a, int b) { // Overloading in the subclass
9          System.out.println("md-Child: " + a + " and " + b);
10     }
11 }
12
13 public class overloading1 {
14     public static void main(String[] args) {
15         Child obj = new Child();
16         obj.md(10);
17         obj.md(10, 20);
18     }
19 }
```

2

3

```
md-Parent: 10
```

Overloading:

same name

different parameters

```
1  class Parent {
2      void md(int a) {
3          System.out.println("md-Parent: " + a);
4      }
5  }
6
7  class Child extends Parent {
8      void md(int a, int b) { // Overloading in the subclass
9          System.out.println("md-Child: " + a + " and " + b);
10     }
11 }
12
13 public class overloading1 {
14     public static void main(String[] args) {
15         Child obj = new Child();
16         obj.md(10);
17         obj.md(10, 20);
18     }
19 }
```

```
md-Parent: 10
md-Child: 10 and 20
```

Overloading:

same name

different parameters

```
1  class OverloadExample {
2      void Mge() {
3          System.out.println("1:No parameters");    }
4
5      void Mge(String message) {
6          System.out.println("2:Message: " + message);    }
7
8      void Mge(String message, int times) {
9          System.out.println("3:Repeated: " + message);    }
10 }
11
12 public class overloading2 {
13     public static void main(String[] args) {
14         OverloadExample obj = new OverloadExample();
15         obj.Mge();
16         obj.Mge("Hello!");
17         obj.Mge("Hello!", 3);
18     }
19 }
```

Overloading:

same name

different parameters

```
1  class OverloadExample {
2      void Mge() {
3          System.out.println("1:No parameters");    }
4
5      void Mge(String message) {
6          System.out.println("2:Message: " + message);    }
7
8      void Mge(String message, int times) {
9          System.out.println("3:Repeated: " + message);    }
10 }
11
12 public class overloading2 {
13     public static void main(String[] args) {
14         OverloadExample obj = new OverloadExample();
15         obj.Mge();
16         obj.Mge("Hello!");
17         obj.Mge("Hello!", 3);
18     }
19 }
```

Overloading:

same name

different parameters

```
1  class OverloadExample {
2      void Mge() {
3          System.out.println("1:No parameters");    }
4
5      void Mge(String message) {
6          System.out.println("2:Message: " + message);    }
7
8      void Mge(String message, int times) {
9          System.out.println("3:Repeated: " + message);    }
10 }
11
12 public class overloading2 {
13     public static void main(String[] args) {
14         OverloadExample obj = new OverloadExample();
15         obj.Mge();
16         obj.Mge("Hello!");
17         obj.Mge("Hello!", 3);
18     }
19 }
```

1
2

```
1:No parameters
```

Overloading:

same name

different parameters

```
1  class OverloadExample {
2      void Mge() {
3          System.out.println("1:No parameters");    }
4
5      void Mge(String message) {
6          System.out.println("2:Message: " + message);    }
7
8      void Mge(String message, int times) {
9          System.out.println("3:Repeated: " + message);    }
10 }
11
12 public class overloading2 {
13     public static void main(String[] args) {
14         OverloadExample obj = new OverloadExample();
15         obj.Mge();
16         obj.Mge("Hello!");
17         obj.Mge("Hello!", 3);
18     }
19 }
```

1:No parameters

Overloading:

same name

different parameters

```
1  class OverloadExample {
2      void Mge() {
3          System.out.println("1:No parameters");    }
4
5      void Mge(String message) {
6          System.out.println("2:Message: " + message);    }
7
8      void Mge(String message, int times) {
9          System.out.println("3:Repeated: " + message);    }
10 }
11
12 public class overloading2 {
13     public static void main(String[] args) {
14         OverloadExample obj = new OverloadExample();
15         obj.Mge();
16         obj.Mge("Hello!");
17         obj.Mge("Hello!", 3);
18     }
19 }
```

```
1:No parameters
2:Message: Hello!
```

Overloading:

same name

different parameters

```
1  class OverloadExample {
2      void Mge() {
3          System.out.println("1:No parameters");    }
4
5      void Mge(String message) {
6          System.out.println("2:Message: " + message);    }
7
8      void Mge(String message, int times) {
9          System.out.println("3:Repeated: " + message);    }
10 }
11
12 public class overloading2 {
13     public static void main(String[] args) {
14         OverloadExample obj = new OverloadExample();
15         obj.Mge();
16         obj.Mge("Hello!");
17         obj.Mge("Hello!", 3);
18     }
19 }
```

```
1:No parameters
2:Message: Hello!
```

Overloading:

same name

different parameters

```
1  class OverloadExample {
2      void Mge() {
3          System.out.println("1:No parameters");    }
4
5      void Mge(String message) {
6          System.out.println("2:Message: " + message);    }
7
8      void Mge(String message, int times) {
9          System.out.println("3:Repeated: " + message);    }
10 }
11
12 public class overloading2 {
13     public static void main(String[] args) {
14         OverloadExample obj = new OverloadExample();
15         obj.Mge();
16         obj.Mge("Hello!");
17         obj.Mge("Hello!", 3);
18     }
19 }
```

```
1:No parameters
2:Message: Hello!
3:Repeated: Hello!
```